

Process Description and Control

Chapter 3

Outline

- ▶ **(Classical) Processes**
- ▶ **Process States**
- ▶ **Process Descriptors**
- ▶ **Process Control**

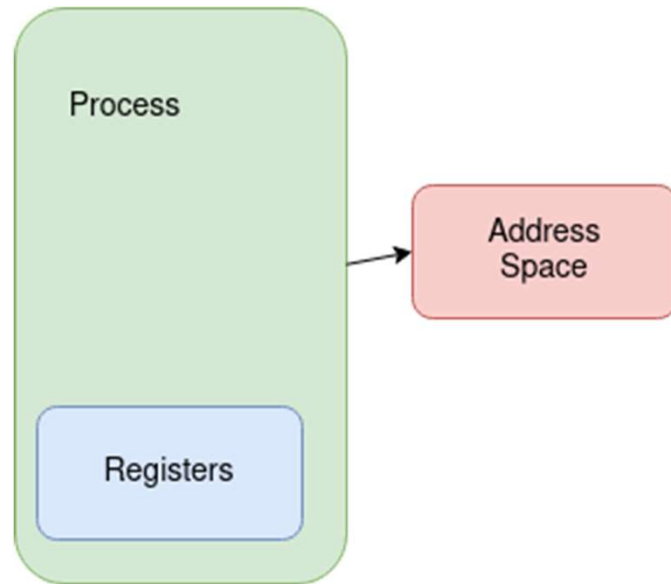


Outline

- ▶ **(Classical) Processes**
- ▶ Process States
- ▶ Process Descriptors
- ▶ Process Control



The Process Abstraction



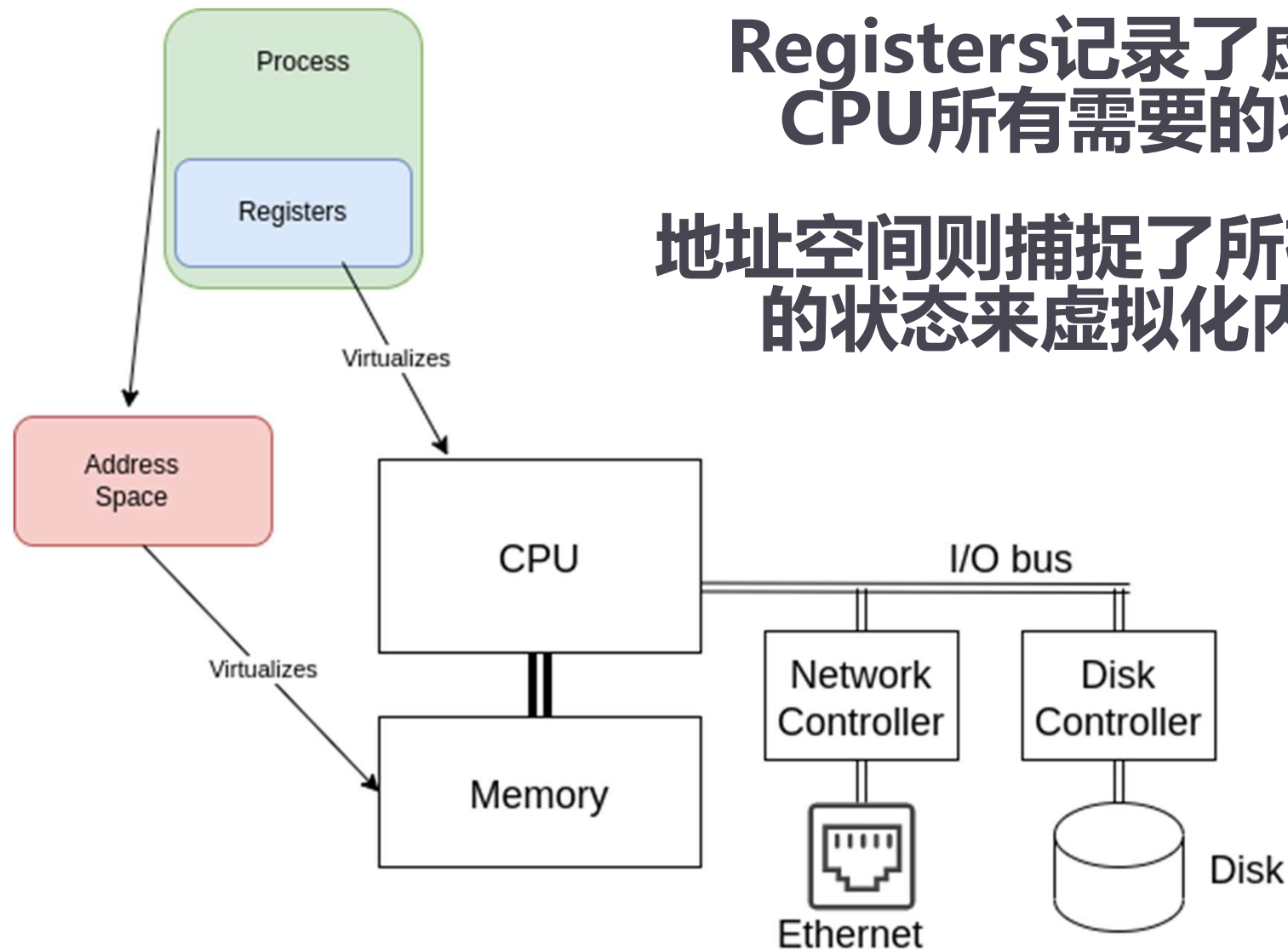
mm_struct 含有这个
进程的内存视图

thread_struct 则是保
存寄存器值的空间

```
// Defined in include/linux/sched.h
struct task_struct {
    //...
    struct mm_struct      *mm;
    // ...
    /* CPU-specific state of this task: */
    struct thread_struct thread;
}
```



The Process Abstraction



Registers记录了虚拟化CPU所有需要的状态

地址空间则捕捉了所有必需的状态来虚拟化内存

Processes

- ▶ Processes support the ability to have (pseudo) concurrent operation
 - ▶ even when there is only one CPU available
 - ▶ The CPU switches back and forth from process to process quickly -- called "multiprogramming"
- ▶ Processes turn a single CPU into multiple virtual CPUs.
 - ▶ They make a modern computer do several things at the same time



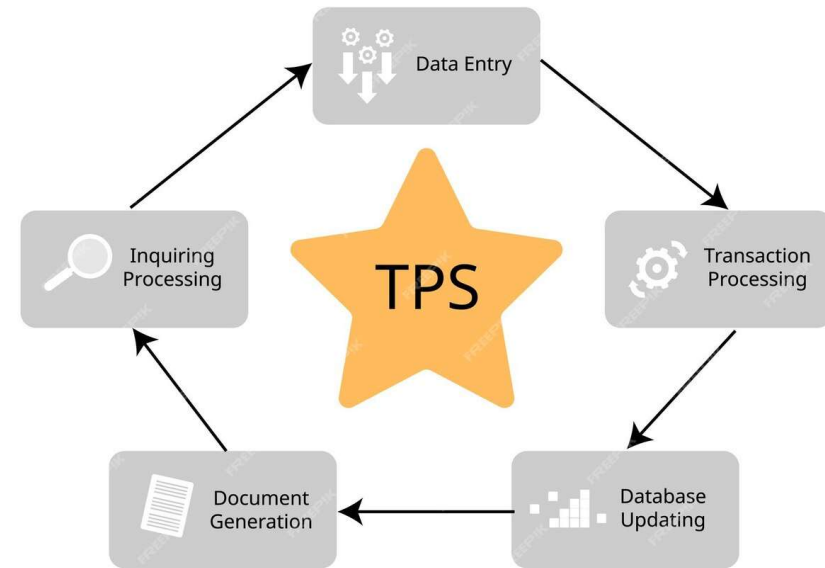
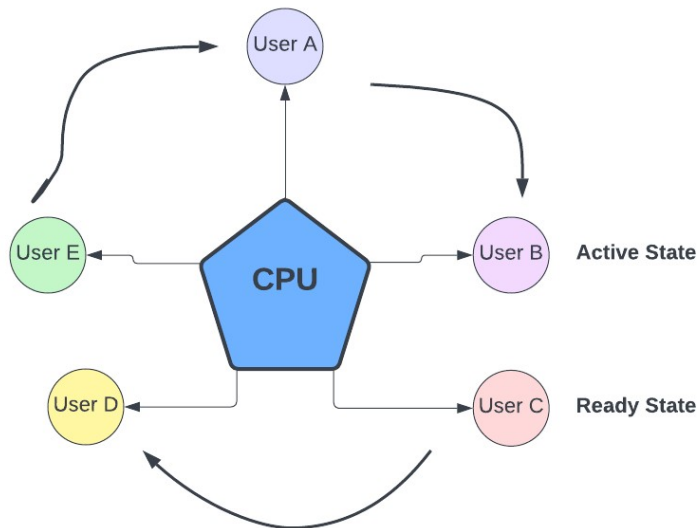
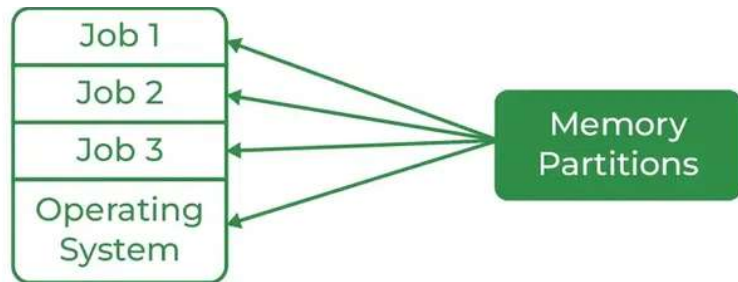
Process Execution

- ▶ Types of *Concurrency* (并发性)
 - ▶ On a uniprocessor: the execution of multiple processes can be interleaved in time (交替执行) → *pseudoparallelism* (伪并行)
 - ▶ On a multiprocessor: interleaved execution + *simultaneous/parallel execution* (并行)
- ▶ Concurrency leads to a host of difficult problems



Multiprogramming OSeS

Multiprogramming batch system



Real-time transaction processing system

Time-sharing system

What are the differences between processes and programs?

Definitions

- ▶ Several Definitions have been given for the term “(classical) process” :
 - ▶ *A program in execution*
 - ▶ *An instance of a program running on a computer*
 - ▶ *The entity that can be assigned to and executed on a processor*
 - ▶ *A unit of activity characterized by a single sequential thread of execution, a current state, and an associated set of system resources*

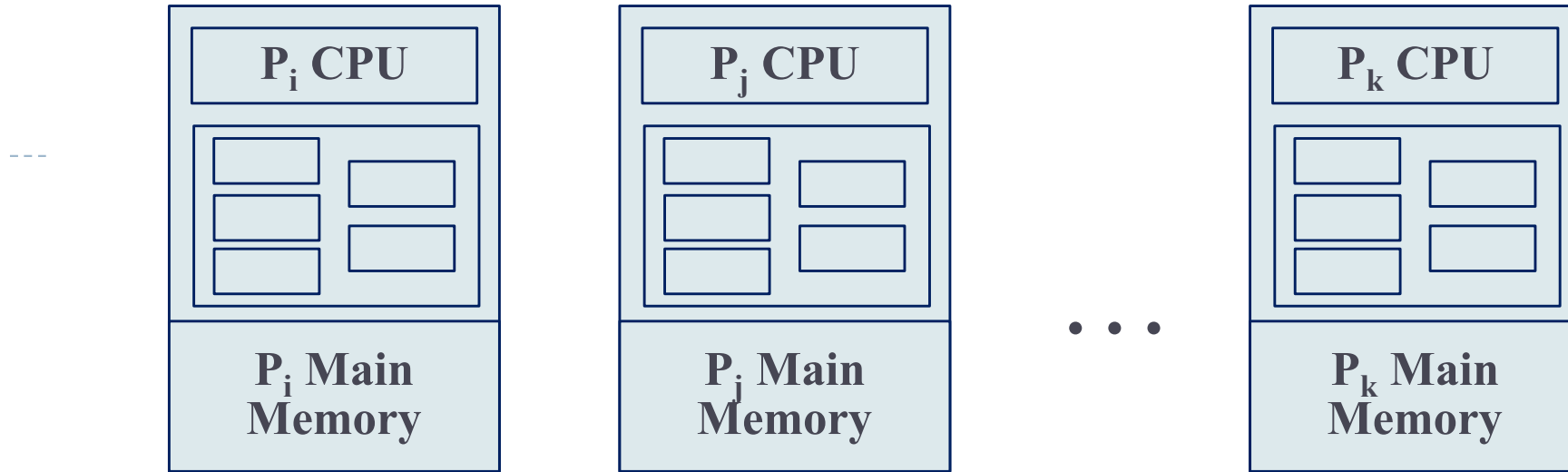
进程的特性：动态性、并发性、独立性、异步性



The Abstract Machine for Classic Processes

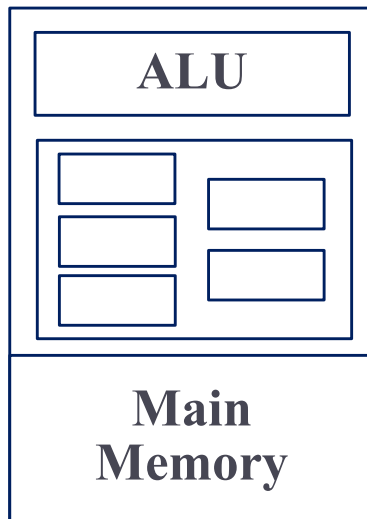
- ▶ **Multiprogramming OSes make the illusion that application programs each have their own exclusive machine where to execute their codes**
- ▶ The OS allocates blocks of main memory using a *space-multiplexing* (空分复用) approach
- ▶ It allocates the processor different classical processes using *time-multiplexing* (时分复用)





OS Interface

CPU

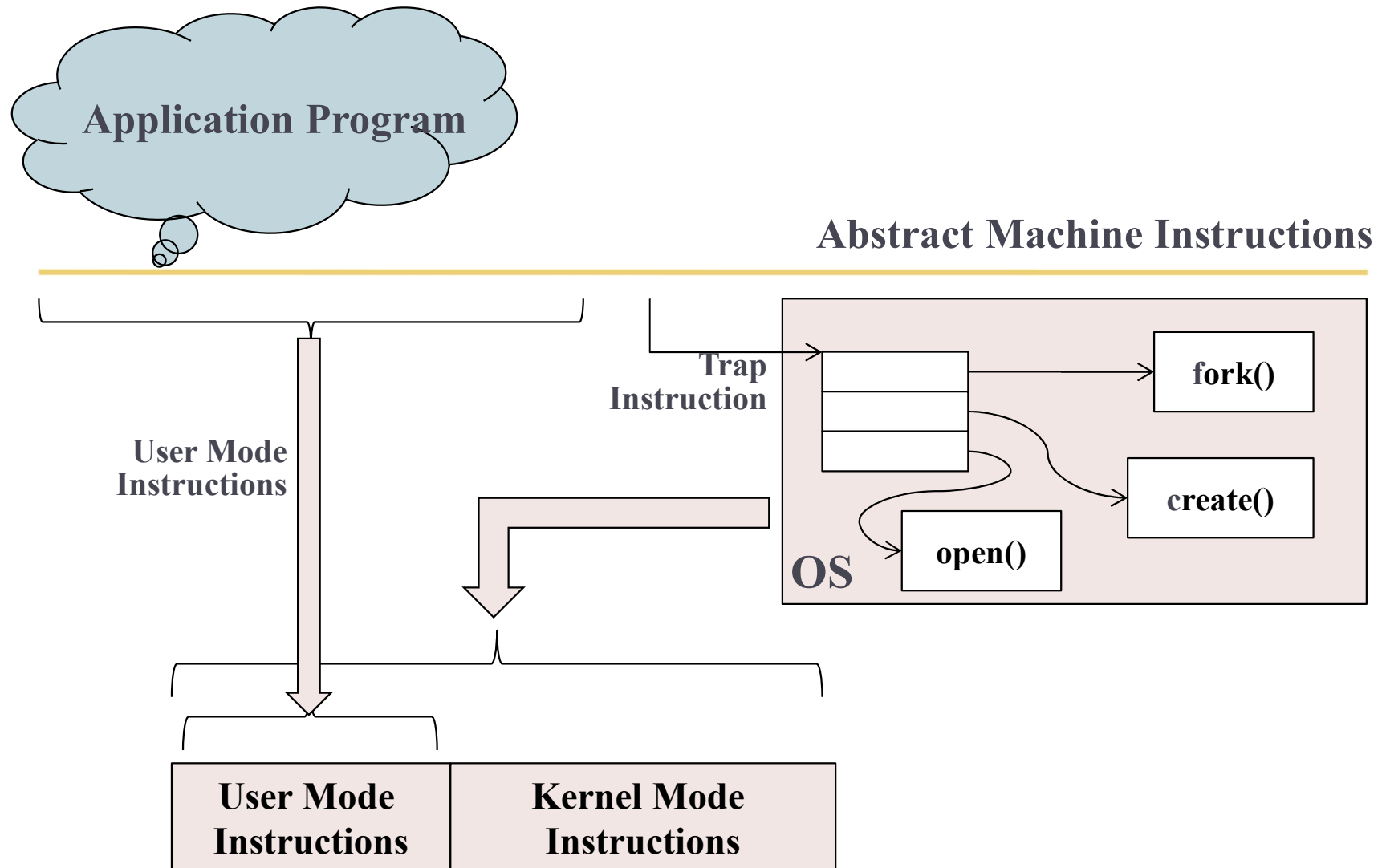


Abstract Machine Interface

- ▶ The abstract machine interface consists of
 - ▶ the **OS system calls**, and
 - ▶ the **user mode instructions** (including trap instruction)
- ▶ The user mode trap instruction
 - ▶ switches the CPU to kernel mode, then branches to an OS function entry point.



Abstract Machine Interface

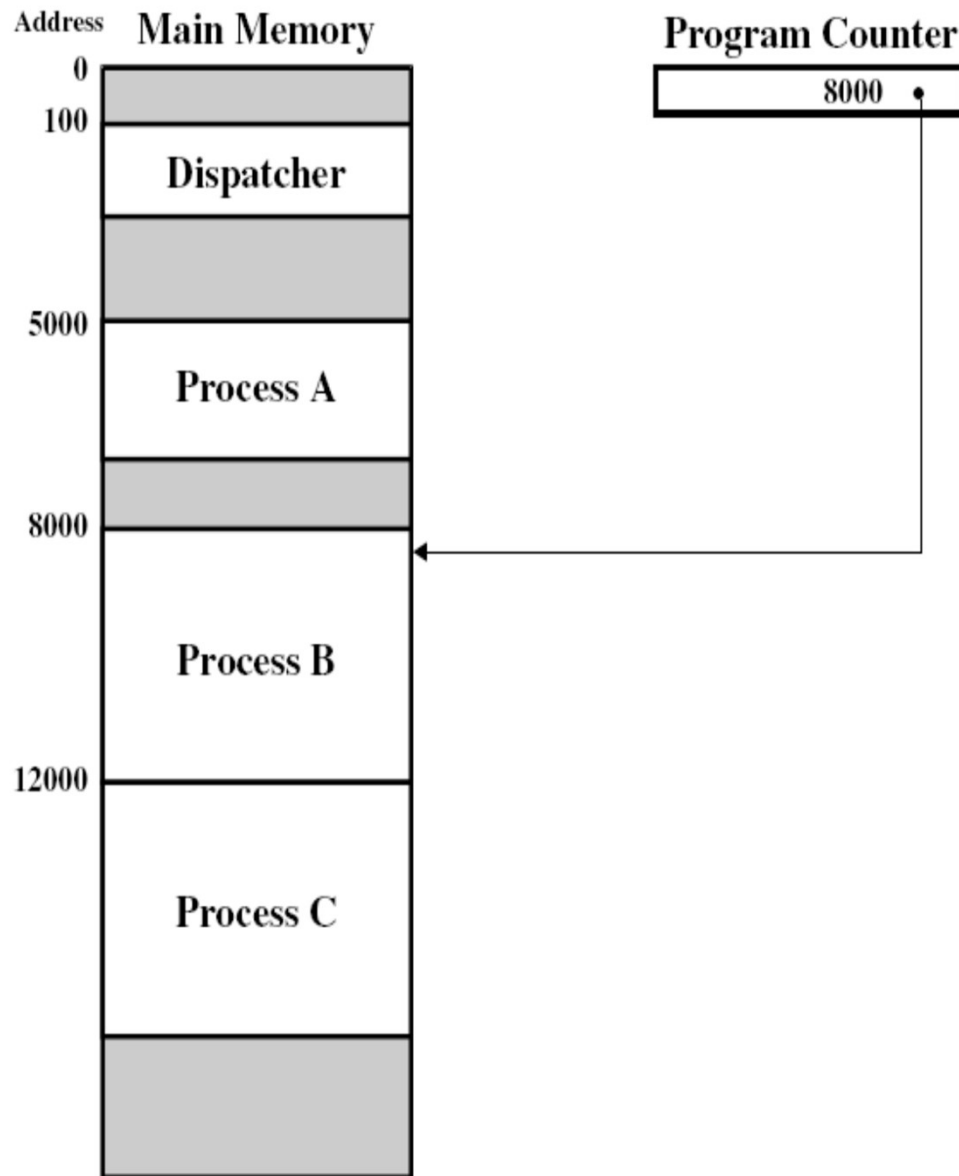


Instruction Trace

指令轨迹

- ▶ **Instruction trace** is used to characterize the behavior of an individual process.
- ▶ Instruction trace for a process is **the sequence of instructions that are executed** for that process.
- ▶ The traces of the various processes are **interleaved**, from the processor's point of view.





There are three processes in system: Process A, Process B, Process C.

**Figure 3.1 Snapshot of Example Execution (Figure 3.3)
at Instruction Cycle 13**

5000	8000	12000
5001	8001	12001
5002	8002	12002
5003	8003	12003
5004		12004
5005		12005
5006		12006
5007		12007
5008		12008
5009		12009
5010		12010
5011		12011

an I/O operation

(a) Trace of Process A

(b) Trace of Process B

(c) Trace of Process C

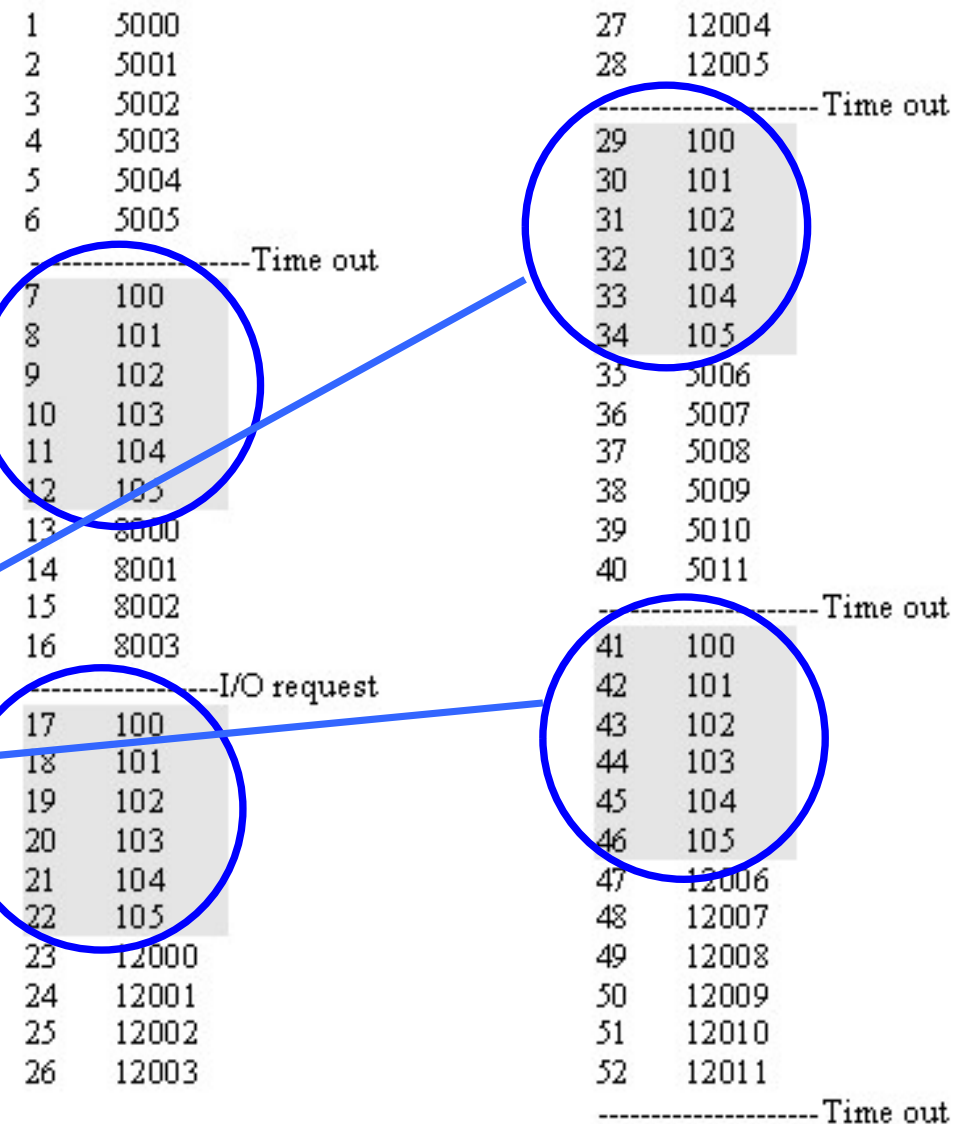
5000 = Starting address of program of Process A
8000 = Starting address of program of Process B
12000 = Starting address of program of Process C

**The OS only allows a process to
continue execution for a
maximum of 6 instructions**

Figure 3.2 Traces of Processes of Figure 3.1

Assume that the OS only allows a process to continue execution for a **maximum of six instruction cycles**

The execution of some code in the dispatcher (分派程序/分派器)



100 = Starting address of dispatcher program

shaded areas indicate execution of dispatcher process;

first and third columns count instruction cycles;

second and fourth columns show address of instruction being executed

Figure 3.3 Combined Trace of Processes of Figure 3.1

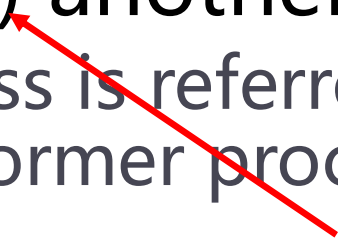
Outline

- ▶ (Classical) Processes
- ▶ **Process States**
- ▶ Process Descriptors
- ▶ Process Control



Process Creation: When?

- ▶ Initialization of a **batch job** (in batch environment)
- ▶ User **logs on** or requests to create a process(in interactive environment)
- ▶ **Created by OS to provide a service** (such as printing)
- ▶ A process can make an explicit request to creates (or **spawns**) another process.
 - ▶ The spawned process is referred to as the child process, while the former process is the parent process



When OS creates a process at the explicit request of another process, the action is referred to as process spawning (进程派生)



Process Termination: When?

- ▶ A batch job issues an **explicit operating system service call** to indicate its completion (**voluntary**) – `exit()` in UNIX and `ExitProcess()` in Windows
- ▶ For an interactive application, the action of the user will indicate when the process is completed:
 - ▶ In a timesharing system: User **logs off or turns off** the terminal
 - ▶ On a personal computer or workstation, **user may quit an application**
- ▶ A number of **error and fault** conditions
- ▶ In some operating systems, a process may be terminated by its **parent process** or when the parent process is terminated. (**involuntary**)



Reasons for Process Termination (1)

- ▶ Time limit exceeded (超过时限)
- ▶ Memory unavailable (无可可用内存)
- ▶ Bounds violation (越界)
- ▶ Protection error (保护错误)
 - ▶ For example: write to read-only file
- ▶ Arithmetic error (算术错误, 例如除以0)
- ▶ Time overrun (时间超出)
 - ▶ process waited longer than a specified maximum for an event



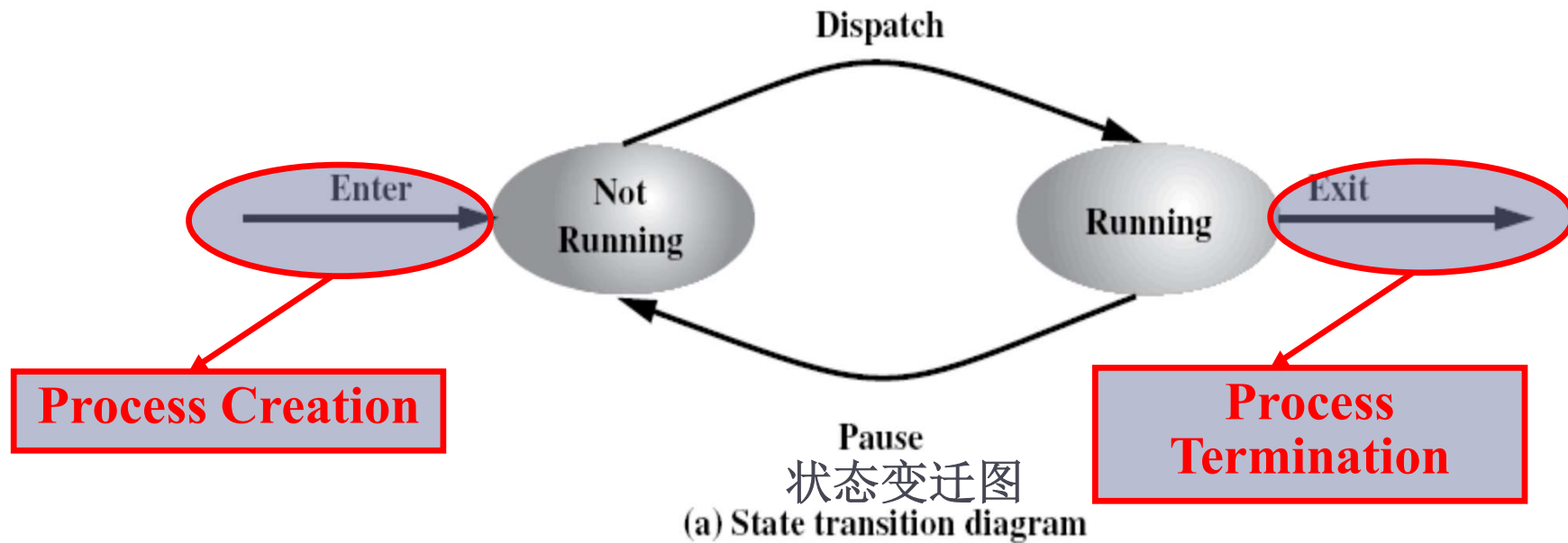
Reasons for Process Termination (2)

- ▶ I/O failure (I/O 失败)
- ▶ Invalid instruction (无效指令)
 - ▶ happens when try to execute data
- ▶ Privileged instruction (特权指令)
- ▶ Operator or OS intervention (操作员或操作系统干预)
 - ▶ such as when deadlock occurs
- ▶ Parent termination (父进程终止)
- ▶ Parent request (父进程请求)

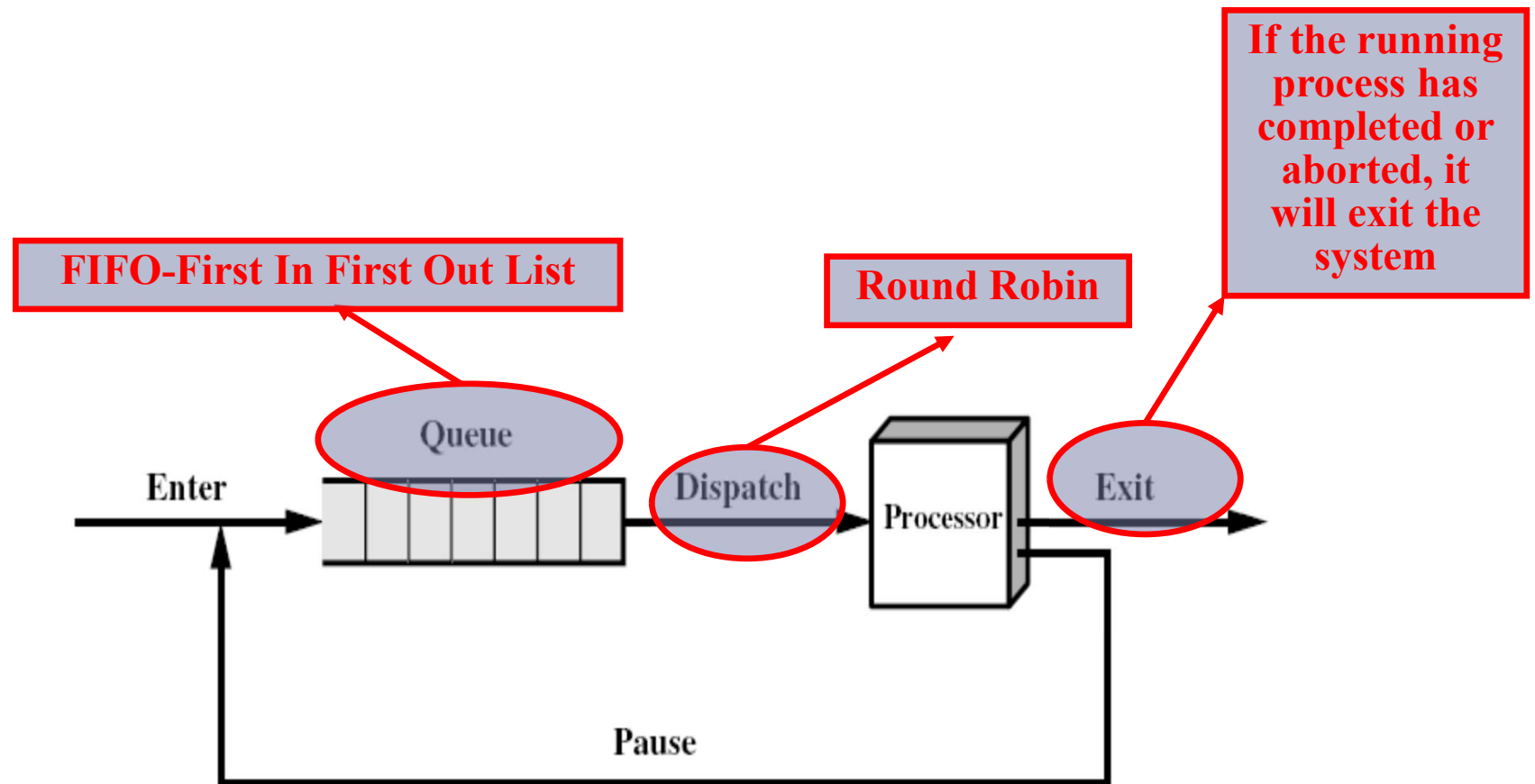


A Two-State Process Model

- ▶ Process may be in one of two states
 - ▶ Running
 - ▶ Not-running



Not-Running Process in a Queue



(b) Queuing diagram

排队图

Dispatch: 分派

Dispatcher: 分派程序/分派器

Two-State Model is Not Enough!

Why?

- ▶ With only one queue of not-running, dispatcher would have to look for the process that is not blocked
 - ▶ cannot just select the process that has been in the queue the longest because it may be blocked
- ▶ Solution: **Split the Not-Running state** into
 - ▶ Ready State
 - ▶ ready to execute
 - ▶ Blocked State
 - ▶ waiting for some event to happen

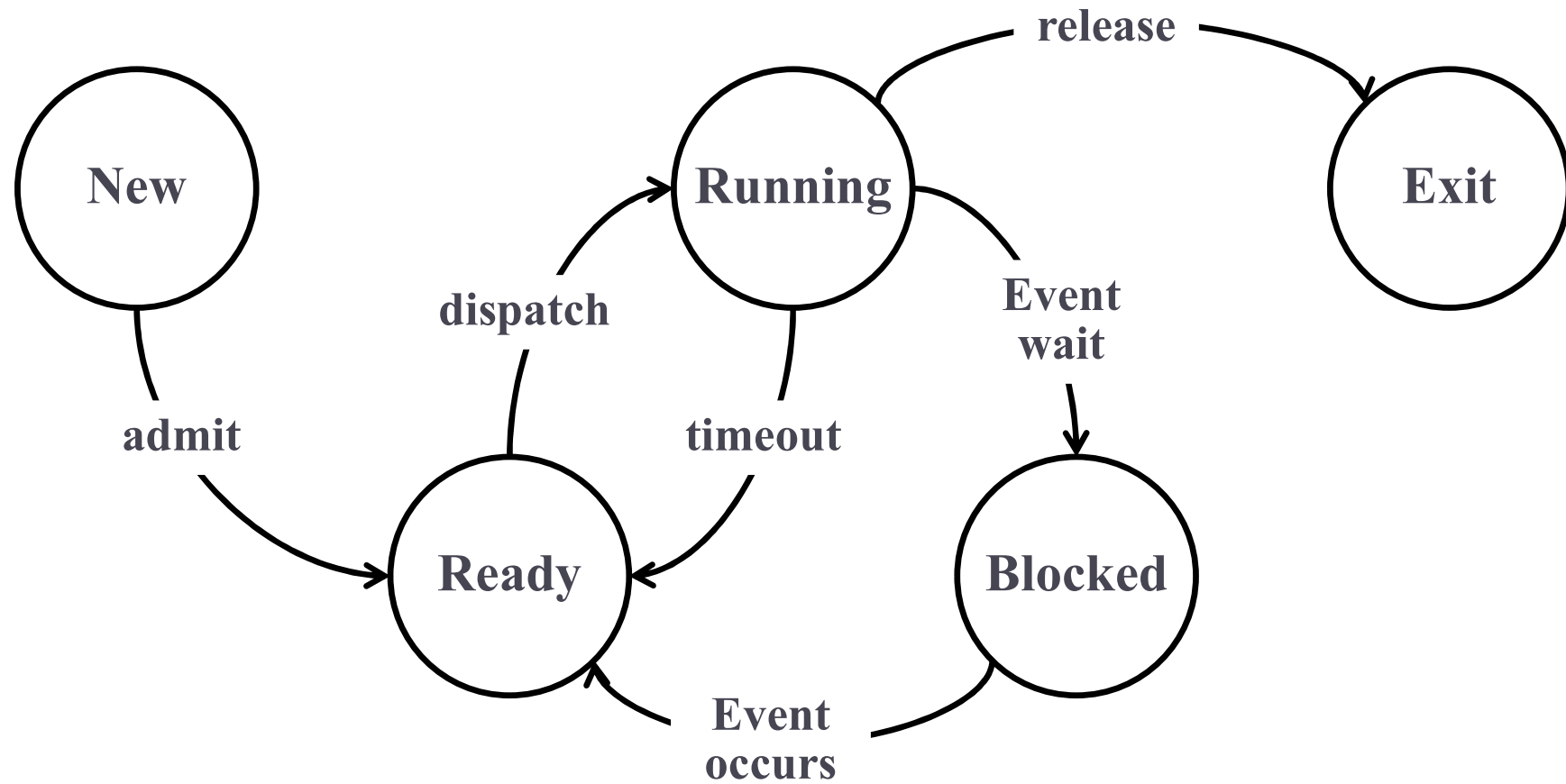


A Five-State Model

- ▶ **Running (运行)**
 - ▶ The process that is currently being executed
- ▶ **Ready (就绪)**
 - ▶ The process that is prepared to execute when given the opportunity
- ▶ **Blocked (阻塞)**
 - ▶ A process that cannot execute until some event occurs.
- ▶ **New (新建)**
 - ▶ Typically a new process has not yet been loaded into main memory
- ▶ **Exit (退出)**



Five-state process model



Why Use “New” State?

- ▶ At New state, the necessary **housekeeping chores** (辅助工作) has been performed by the OS
 - ▶ An identifier is associated with the process
 - ▶ Any tables for managing the process are allocated and built
- ▶ The OS may **limit the number of processes** that may be in the system **for reasons of performance or main memory limitation.**
 - ▶ The process itself is not in main memory (the program is still in secondary storage)
 - ▶ In systems that support virtual memory, the program and data are loaded into virtual memory when a process moves from New to Ready



Why Use “Exit” State?

- ▶ Exit State: The tables and other information associated with the process is temporarily preserved by OS
 - ▶ which provides time for auxiliary or support programs to extract any needed information
 - ▶ An accounting program → processor time and other resource utilization → billing purposes
 - ▶ A utility program → information about the history of the process → performance or utilization analysis



Possible State-Transitions (1)

- ▶ Null→New: A process is created
- ▶ New→Ready: The OS moves a process from New state to Ready state, when it is prepared to take on an additional process
- ▶ Ready→Running: When it is time to select a new process to run
- ▶ Running→Exit: normal termination or aborts
- ▶ “Ready→Exit” or “Blocked→Exit”: These two transitions are not shown on the state diagram.
 - ▶ In some systems: A parent can terminate a child process at any time (or the termination of a parent process will cause the termination of all its child processes)



Possible State-Transitions (2)

▶ Running → Ready

- ▶ The running process has reached the maximum allowable time
- ▶ A running process is preempted by another process of higher priority level
- ▶ A process may voluntarily release control of the processor

▶ Running → Blocked

- ▶ A process is put in the blocked state if it requests something for which it must wait. (waiting for I/O operation to completion or a message from another process)

▶ Blocked → Ready

- ▶ When the event for which a process in the Blocked state is waiting occurs, the process is moved to the Ready state.



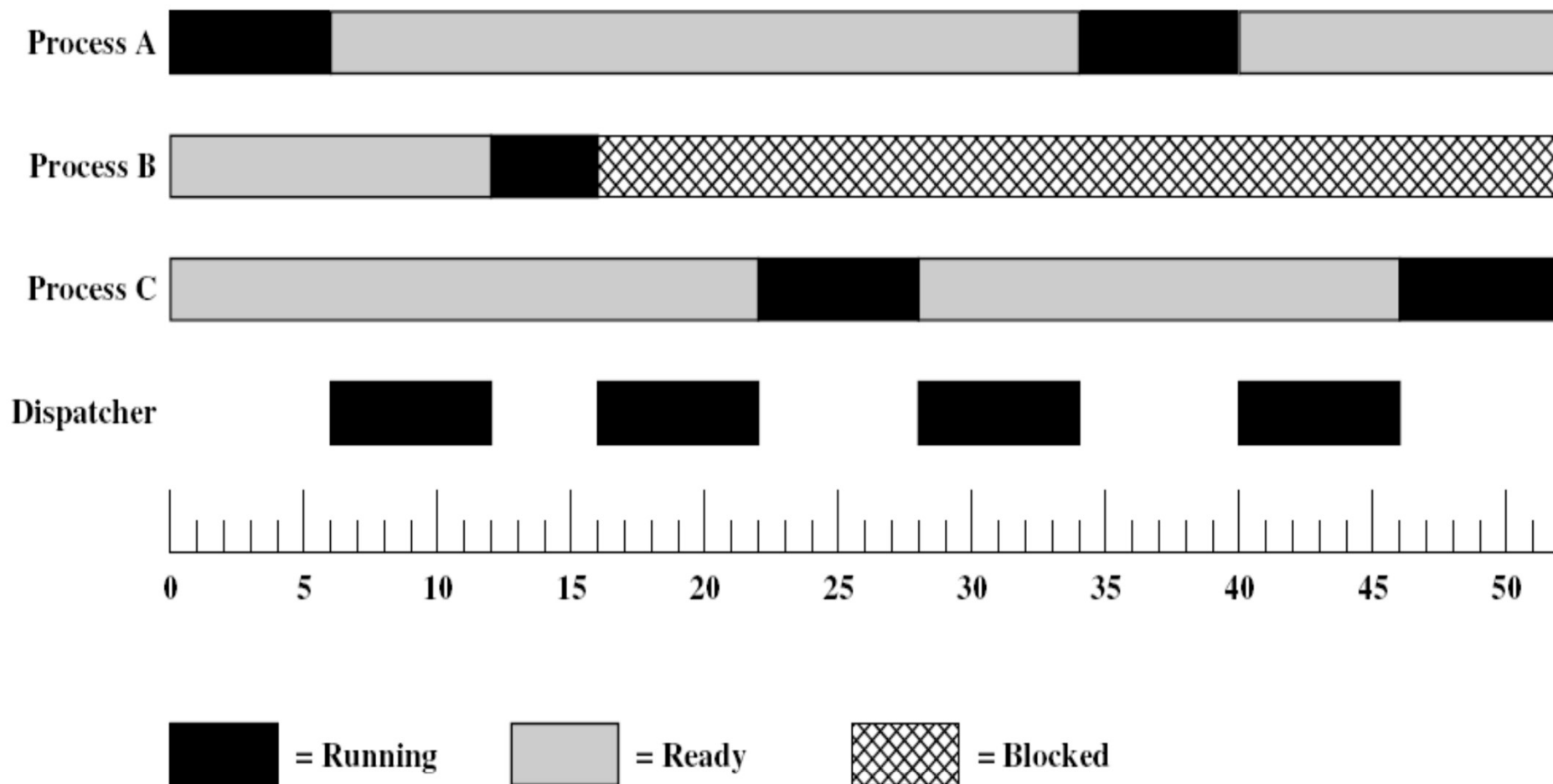
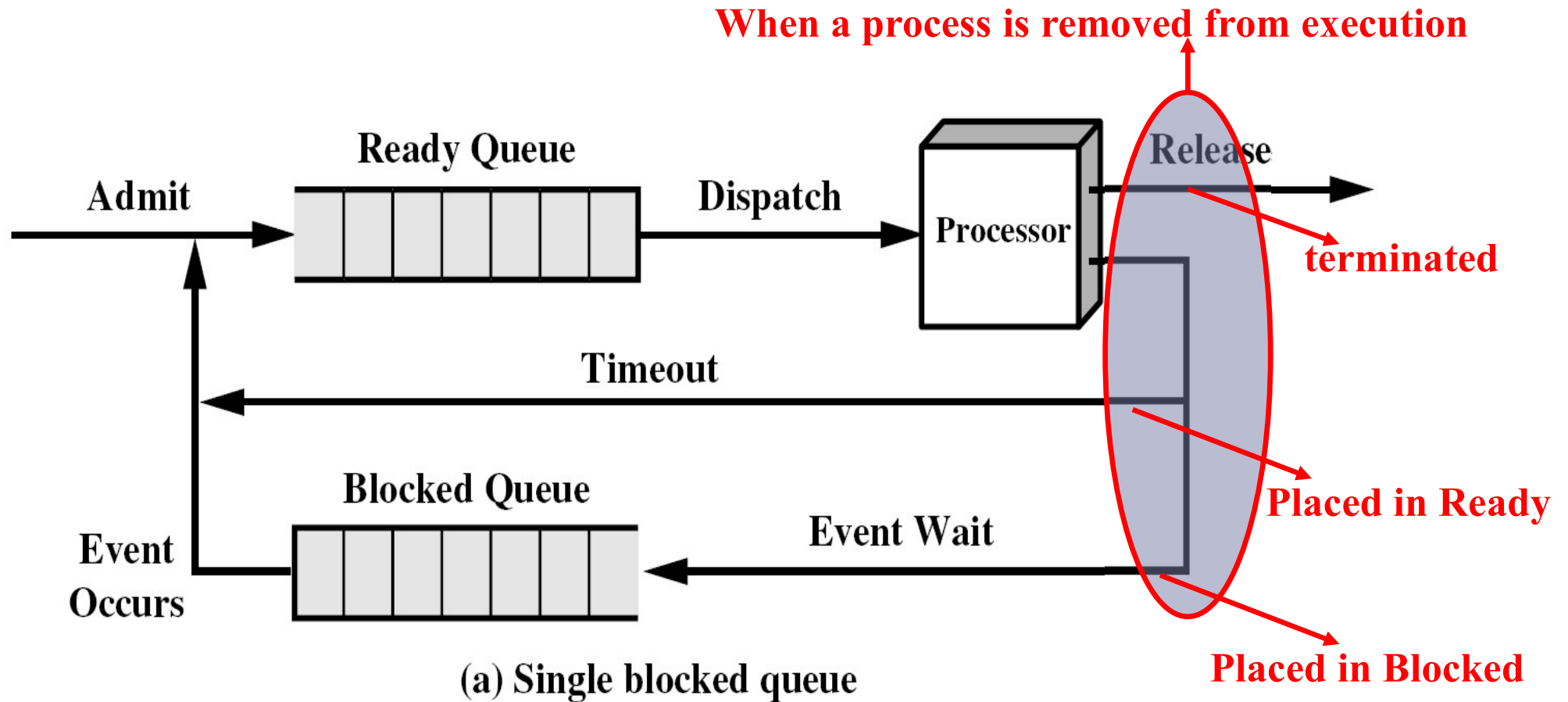


Figure 3.6 Process States for Trace of Figure 3.3

Queuing Models

Single Blocked Queue

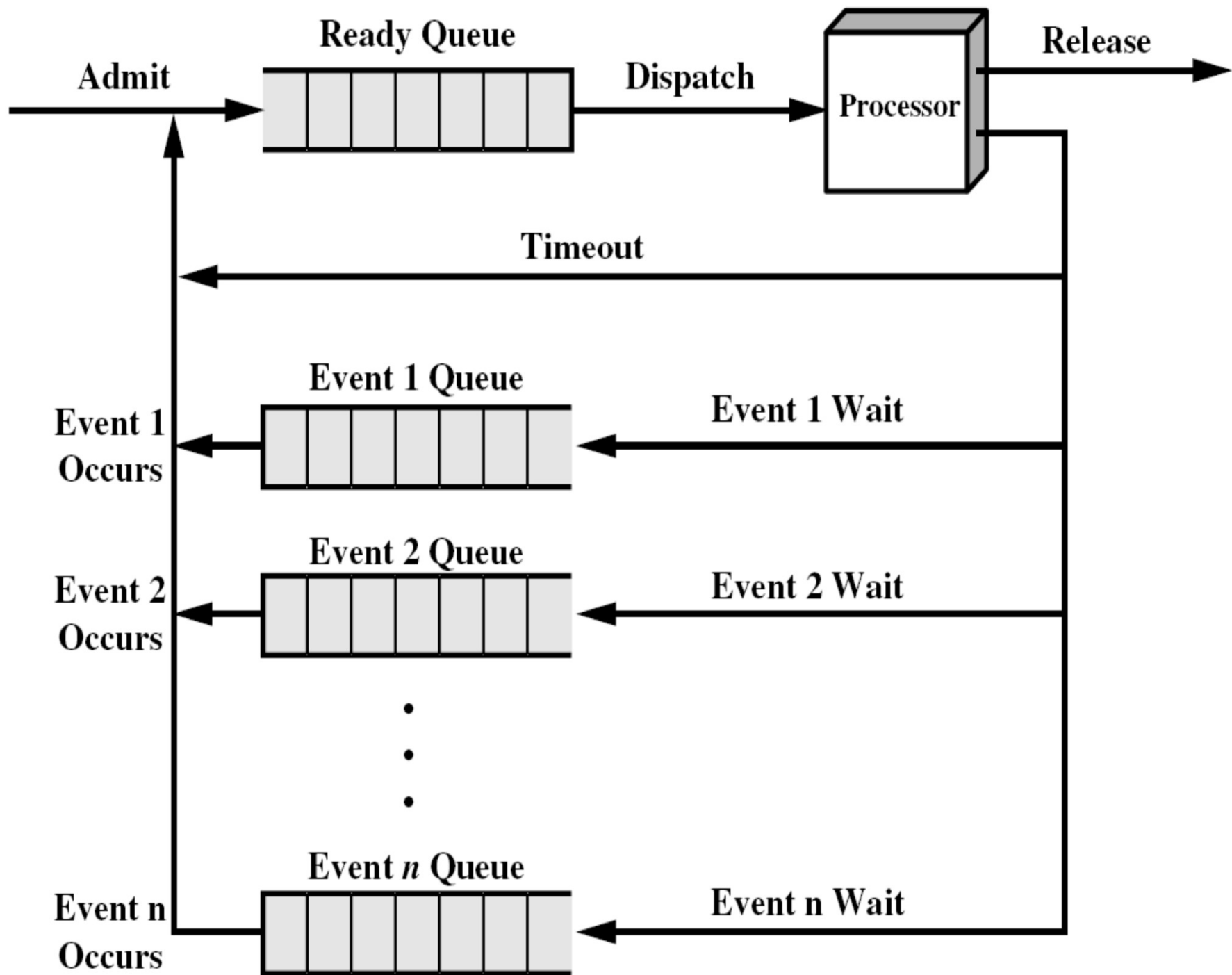


Queuing Model

Multiple Blocked Queues

- ▶ What is the problem with a single Blocked queue?
 - ▶ When an event occurs, the OS must scan the entire Blocked queue to search for those purposes waiting on that event.
 - ▶ In a larger operating system, there could be hundreds or even thousands of processes in that queue → inefficiency!!
- ▶ How about having a number of queues, **one for each event?**





(b) Multiple blocked queues

Final Refinement

Multiple Ready Queues

- ▶ If the dispatching of processes is dictated by a priority scheme, then it would be convenient to have a number of Ready queues!
- ▶ The operating system could easily determine which is the highest-priority ready process that has been waiting the longest.
- ▶ **Why????**



Suspended Processes (1)

挂起的进程

It should be blamed !!! For not only the Suspended States but also the Blocked State

- ▶ Let us consider **a system without employing virtual memory.**
 - ▶ Each process to be executed must be loaded **fully** into main memory.
- ▶ Processor is so much faster than I/O that the processor could be still idle most of the time!! **How to improve the utilization of the processor??**
 - ▶ To **expand the main memory** to accommodate more processes????
 - ▶ To **Swap these processes to disk** to free up more memory



Suspended Processes (2)

Swapping (交换)

- ▶ When none of the processes in main memory is in the Ready state, the OS swaps one of the blocked processes out onto disk (**into a suspend state**)
 - ▶ Correspondingly, there is a suspend queue.
- ▶ The OS then brings in **another process from the suspend queue**, or **accepts a new-process request**.
- ▶ Swapping is also an I/O operation. Why it does not make the situation worse?
 - ▶ Disk I/O is the fastest I/O on a system!!



The Design of Suspend States

▶ One Suspend State

▶ The difficulty is:

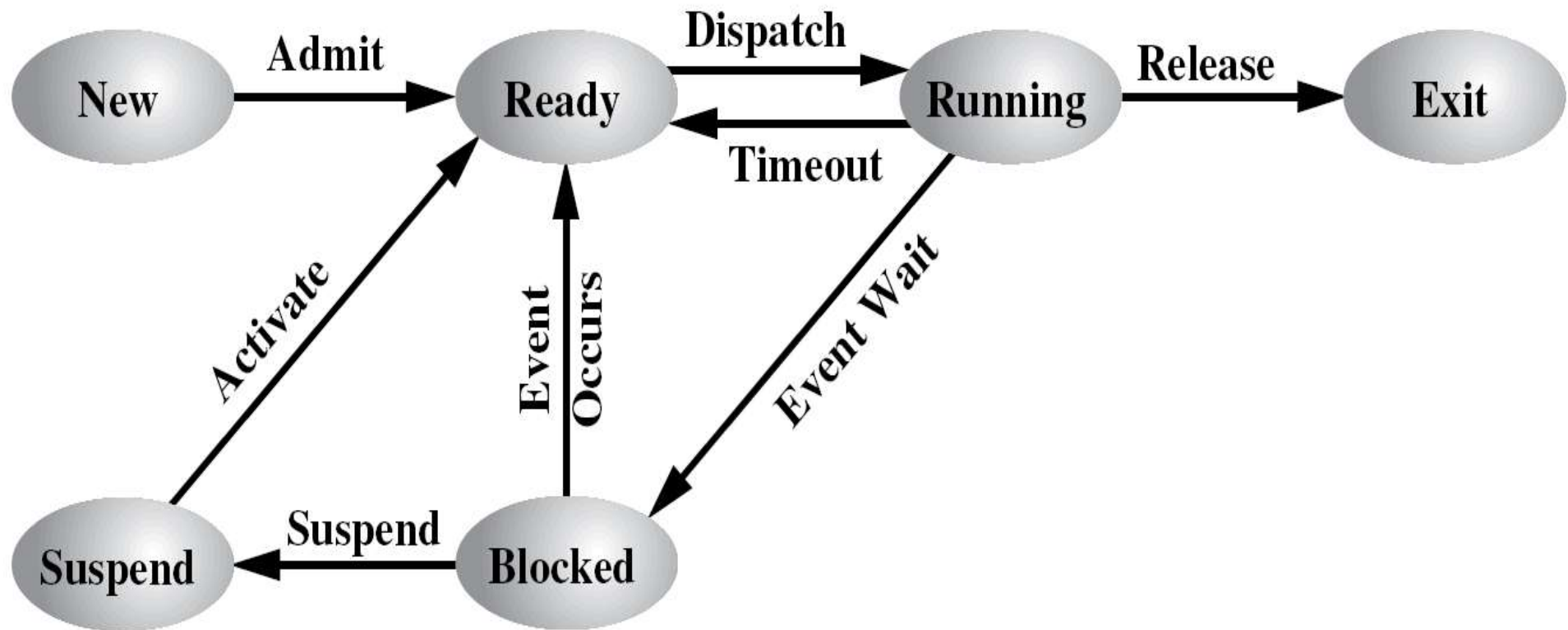
- ▶ All of the processes that have been suspended were in the Blocked state. (**It does no good to bring a blocked process back into main memory**)
- ▶ However, blocked process is waiting for an event. (if that event occurs, the process is not blocked.)

▶ Two Suspend States

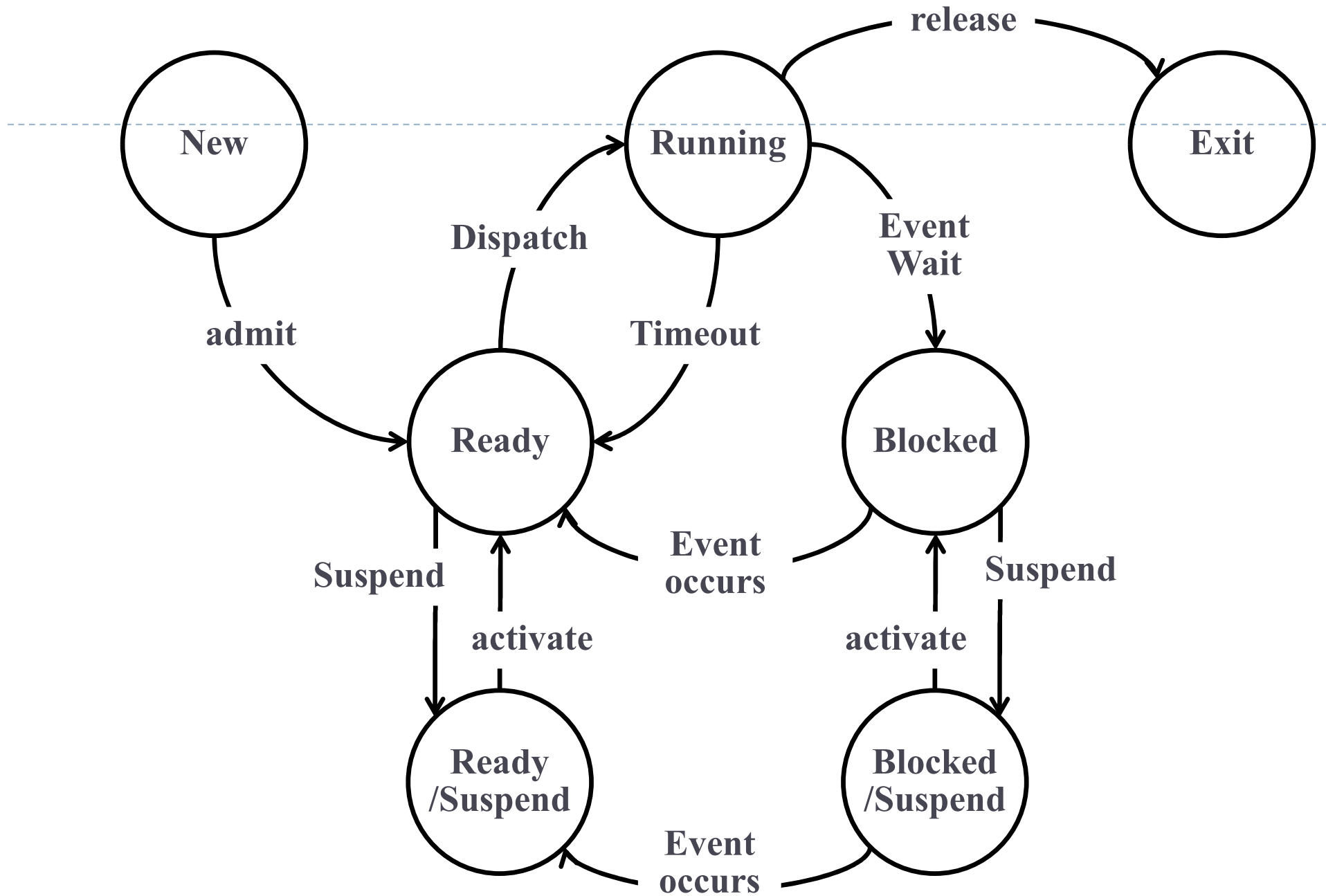
- ▶ Blocked/Suspend: The process is in secondary memory and awaiting an event.
- ▶ Ready/Suspend: The process is in secondary memory and is available for execution if loaded into main memory.



One Suspend State



(a) With One Suspend State



Two Suspend States

Important New Transitions (1)

- ▶ **Blocked→Blocked/Suspend:**
 - ▶ No ready processes
 - ▶ OS determines that the process to be dispatched requires more main memory.
- ▶ **Blocked/Suspend→Ready/Suspend:**
 - ▶ The event that the process is waiting for occurs
- ▶ **Ready/Suspend→Ready**
 - ▶ When there are no ready processes in main memory
 - ▶ A process in the Ready/Suspend has higher priority than any processes in the Ready state.



Important New Transitions (2)

- ▶ Ready→Ready/Suspend: (Normally, OS prefers to suspend a Blocked process)
 - ▶ It may be necessary to suspend a ready process if it is the only way to free up a sufficiently large block of main memory.
 - ▶ The OS may choose to suspend a lower-priority ready process rather than a higher-priority blocked process.



Other Transitions

- ▶ **New→Ready/Suspend and New→Ready:**
- ▶ **Blocked/Suspend→Blocked:**
- ▶ **Running→Ready/Suspend**
- ▶ **Various→Exit:**



Reasons for Process Suspension

Swapping

The operating system needs to release sufficient main memory to bring in a process that is ready to execute.

Other OS reason

The operating system may suspend a background or utility process or a process that is suspected of causing a problem.

Interactive user request

A user may wish to suspend execution of a program for purposes of debugging or in connection with the use of a resource.

Timing

A process may be executed periodically (e.g., an accounting or system monitoring process) and may be suspended while waiting for the next time interval.

Parent process request

A parent process may wish to suspend execution of a descendent to examine or modify the suspended process, or to coordinate the activity of various descendents.

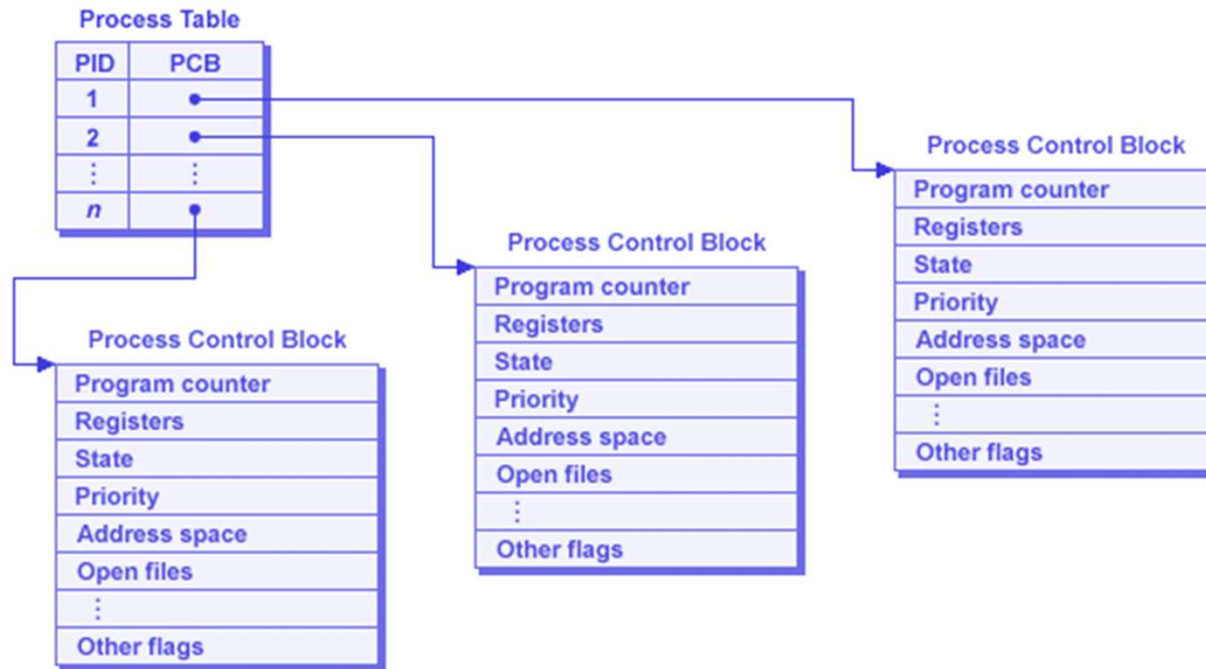


Outline

- ▶ (Classical) Processes
- ▶ Process States
- ▶ **Process Descriptors**
- ▶ Process Control



Process Tables

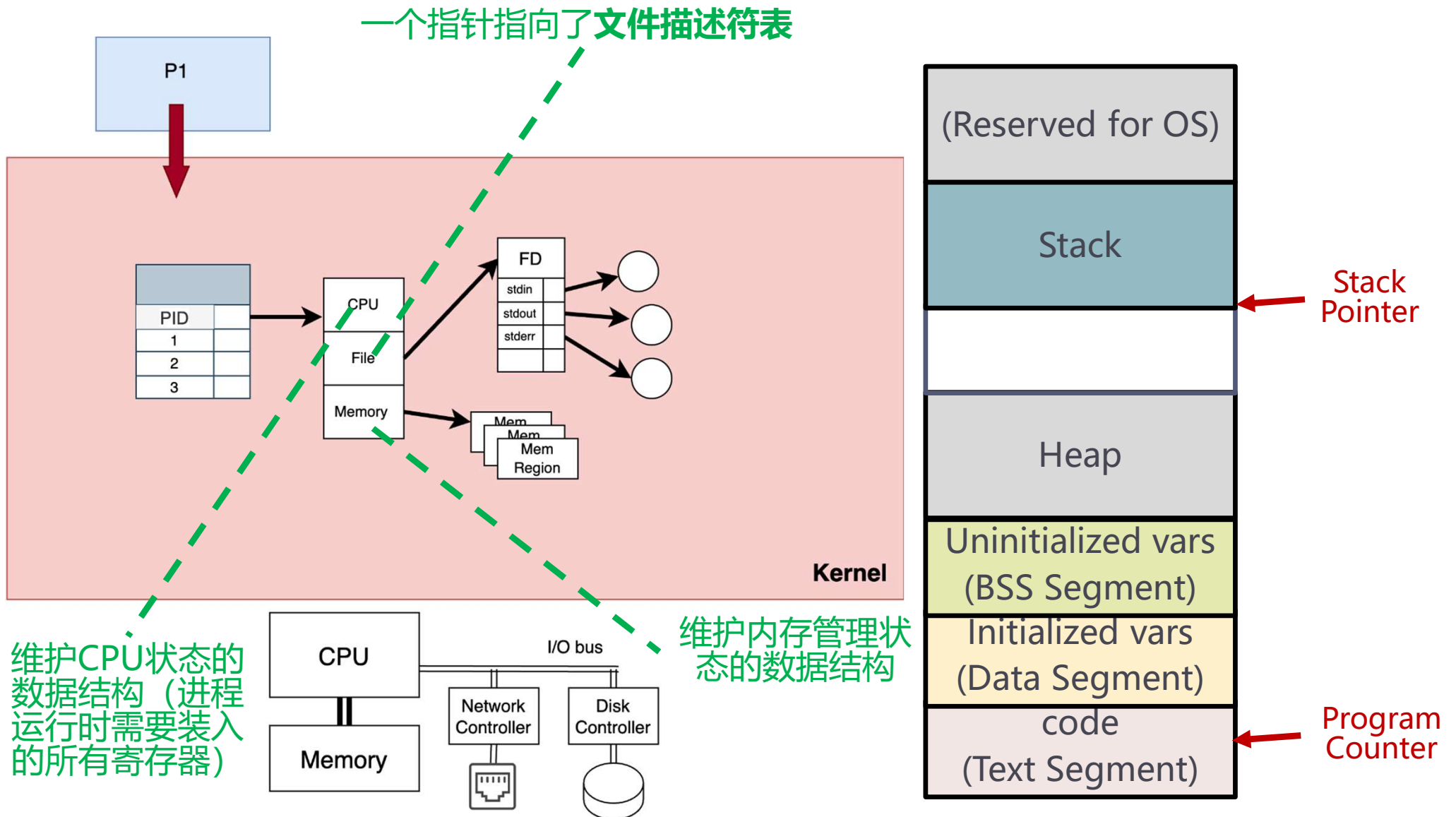


The OS maintains a table (an array of structures), called the **process table**, with one entry per process

The entries are called **process control block** or **process descriptor**



Process: Information



Process Location

- ▶ The location of a process image depends on the memory management scheme being used.
- ▶ When the process image is maintained as a contiguous block of memory.
 - ▶ The operating system must know the location of each process on disk and the location of that process in main memory.
- ▶ When a process image consists of a set of blocks (**page** or **segment** or a **combined** one) that need not be stored contiguously.
 - ▶ The process table must show the location of each block



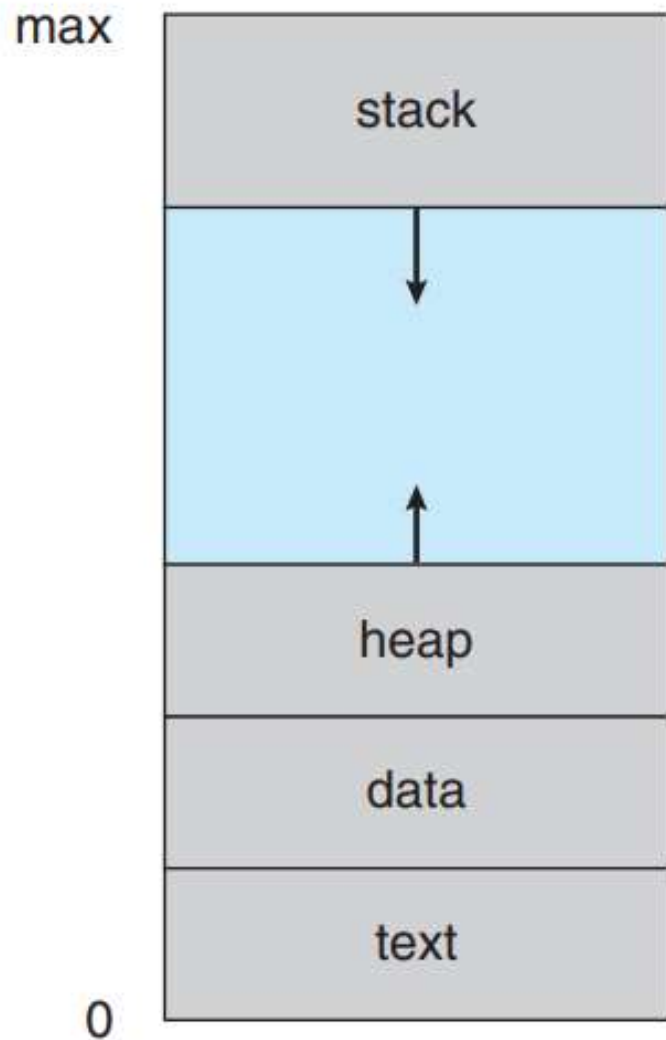
Process Image

进程映像

- ▶ **Process includes set of programs to be executed**
 - ▶ Data locations for local and global variables
 - ▶ Any defined constants
 - ▶ Stack (栈)
 - ▶ Heap (堆)
- ▶ **Process control block**
 - ▶ Collection of attributes about a specific process
- ▶ **Process image**
 - ▶ Collection of program, data, stack, and attributes



Memory Layout of Process



Temporary data storage when invoking functions (such as function parameters, return addresses, and local variables)
(栈段: 函数调用时的临时数据存储)

Memory that is dynamically allocated during program run time (堆段: 动态分配内存)

Global variables (数据段: 全局变量)

Executable code (文本段: 可执行代码)



Process Control Block (PCB)

Why does OS need PCB?

To manage numerous concurrent processes, OS must maintain a data structure for each process, recording key information such as its identity, execution status, and resource usage. This data structure is known as the Process Control Block (PCB) or Process Descriptor; it serves as the unique identifier for a process's existence and forms the foundation for the operating system's process management.

核心结论：PCB 的核心价值

PCB 是操作系统感知和管理进程的**唯一依据**。它将抽象的进程实体化，让系统能够对进程进行创建、调度、暂停和终止等全生命周期的管理，是操作系统内核运作的基石。



Process Control Block Information Organization

The information stored in PCB are usually grouped into three categories:

- ❑ Process Identification (进程标识信息),
- ❑ Processor State Information (处理器状态信息)
- ❑ Process Control Information (进程控制信息)

➤ Process ID

➤ User ID

Process Identification

➤ Saved CPU state

Processor State Information

➤ Execution state

➤ OS resources

➤ Memory management info

➤ Scheduling priority

➤ Accounting information

Process Control Information

Process Control Block: Information Categories

01 进程标识信息

PID (进程ID): 进程的唯一数字身份证，是操作系统调度和管理进程的核心索引。

PPID (父进程ID): 标识进程的创建者，构建系统的进程树结构，维护父子进程的生命周期关联。

UID/GID: 定义进程的用户归属与权限边界，决定其对系统资源的访问能力。

02 CPU状态信息

进程上下文: 进程被中断时CPU寄存器的快照，包含通用寄存器、状态寄存器及栈指针的值。

中断与恢复: 进程切换时保存上下文，恢复执行时重新加载，确保程序执行的连续性。

核心作用: 是实现多任务并发、分时系统及抢占式调度的底层技术基础。

03 进程控制信息

涵盖**进程状态与调度信息、内存管理信息、进程间通信、记账信息、文件和I/O资源信息**等诸多信息，为OS对进程的管理和控制提供信息支撑

总结: 进程标识是进程的“身份凭证”，而CPU状态则是实现并发执行的“记忆载体”。

PCB Information: Process Control Information (1)

- ▶ *Scheduling and State Information*: This information is needed by the OS to perform its scheduling function. Typical items of information:
 - ▶ *Process state*: (e.g., running, ready, waiting).
 - ▶ *Priority*: describes the scheduling priority of the process
 - ▶ *Scheduling-related information*: This will depend on the scheduling algorithm used. (e.g. the amount of time that the process has been waiting and the amount of time that the process executed the last time it was running.
 - ▶ *Event*: Identity of event the process is awaiting before it can be resumed



PCB Information: Process Control Information (2)

- ▶ *Data Structuring*: A process may be linked to other process in a queue, ring, or some other structure. For example,
 - ▶ all processes in a ready state for a particular priority level may be linked in a **queue**.
 - ▶ A process may exhibit a parent-child (creator-created) relationship with another process. → **Tree structure**
 - ▶ The process control block may contain pointers to other processes to support these structures.



PCB Information: Process Control Information (3)

▶ *Interprocess Communication*

- ▶ Various flags, signals, and messages may be associated with communication between two independent processes. Some or all of this information may be maintained in the process control block.



PCB Information: Process Control Information (4)

- ▶ *Memory Management (内存管理)*
 - ▶ include pointers to segment and/or page tables that describe the virtual memory assigned to this process.
- ▶ *Resource Ownership*
 - ▶ Resources controlled by the process may be indicated, such as opened files.
- ▶ *Accounting Information (记账信息):*
 - ▶ A history of utilization of the processor or other resources may also be included; this information may be needed by the scheduler.



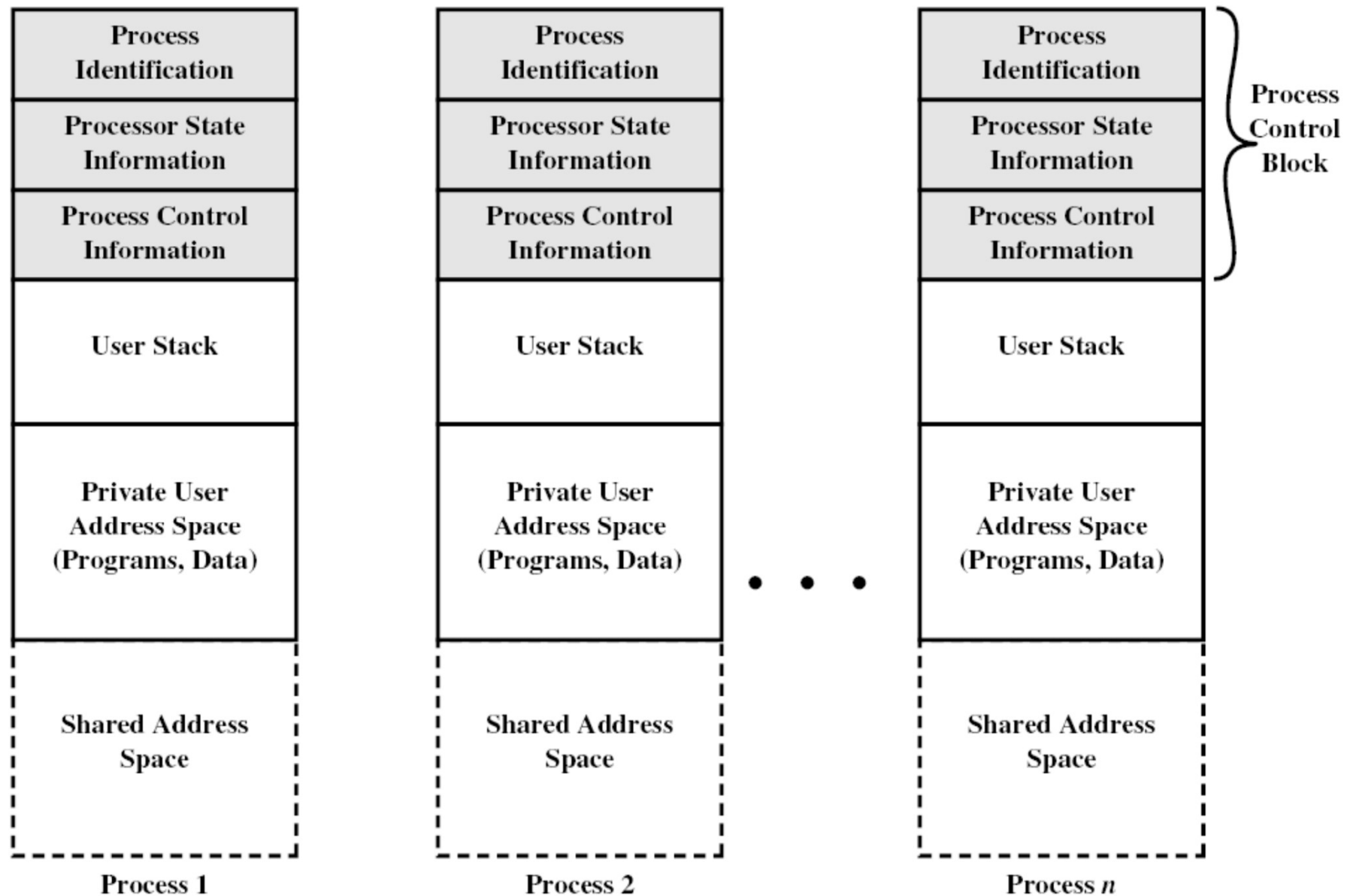
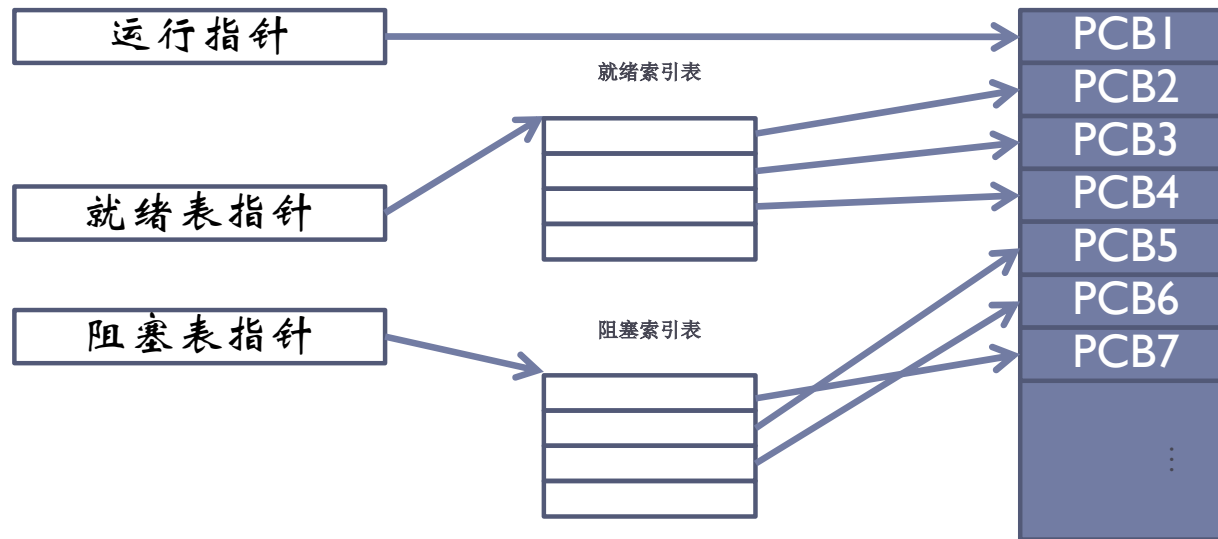
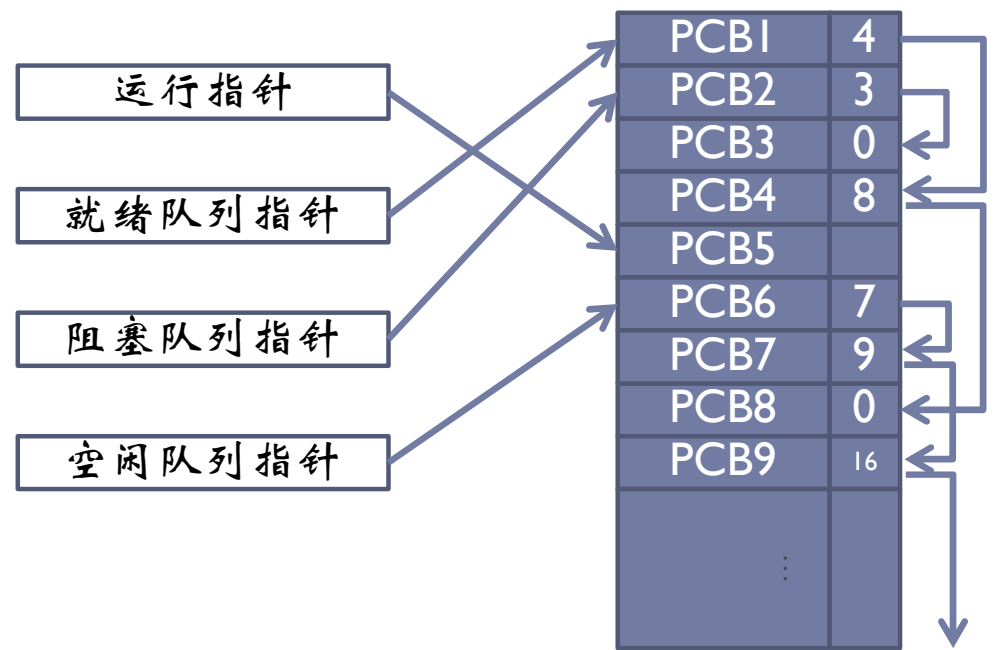


Figure 3.12 User Processes in Virtual Memory

The Organization of PCBs

- ▶ Two common way to manage the PCBs
 - ▶ 链接方式：
 - ▶ 把具有同一状态的PCB，用其中的链接字链接成一个队列，从而形成就绪队列、阻塞队列和空白队列等
 - ▶ 索引方式：
 - ▶ 系统根据进程的状态创建几张索引表，索引表在内存的首地址记录在内存的一些专用单元中。
 - ▶ 索引表的每个表目指向具有相应状态的某个PCB





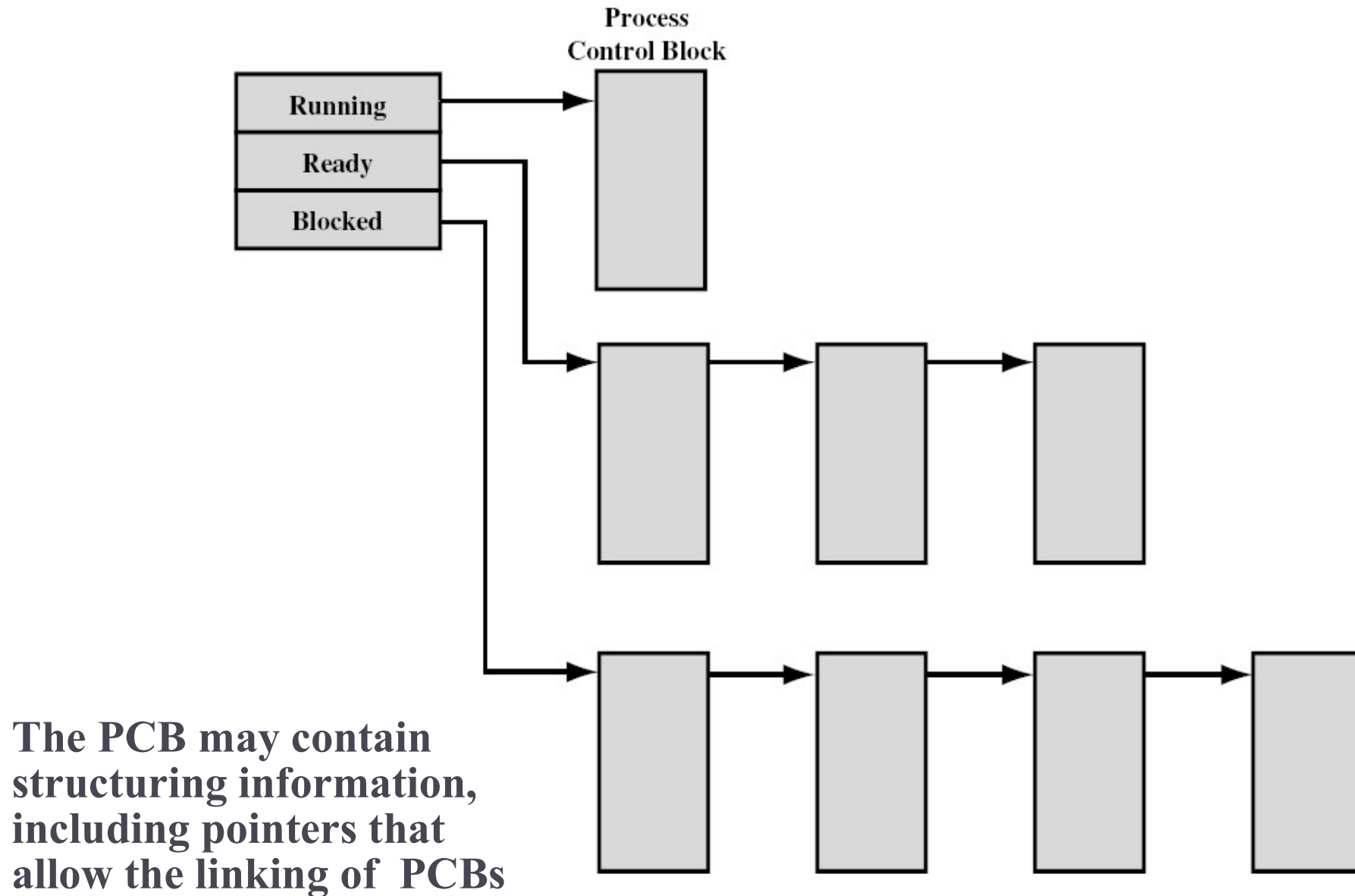


Figure 3.13 Process List Structures

Outline

- ▶ (Classical) Processes
- ▶ Process States
- ▶ Process Descriptors
- ▶ **Process Control**

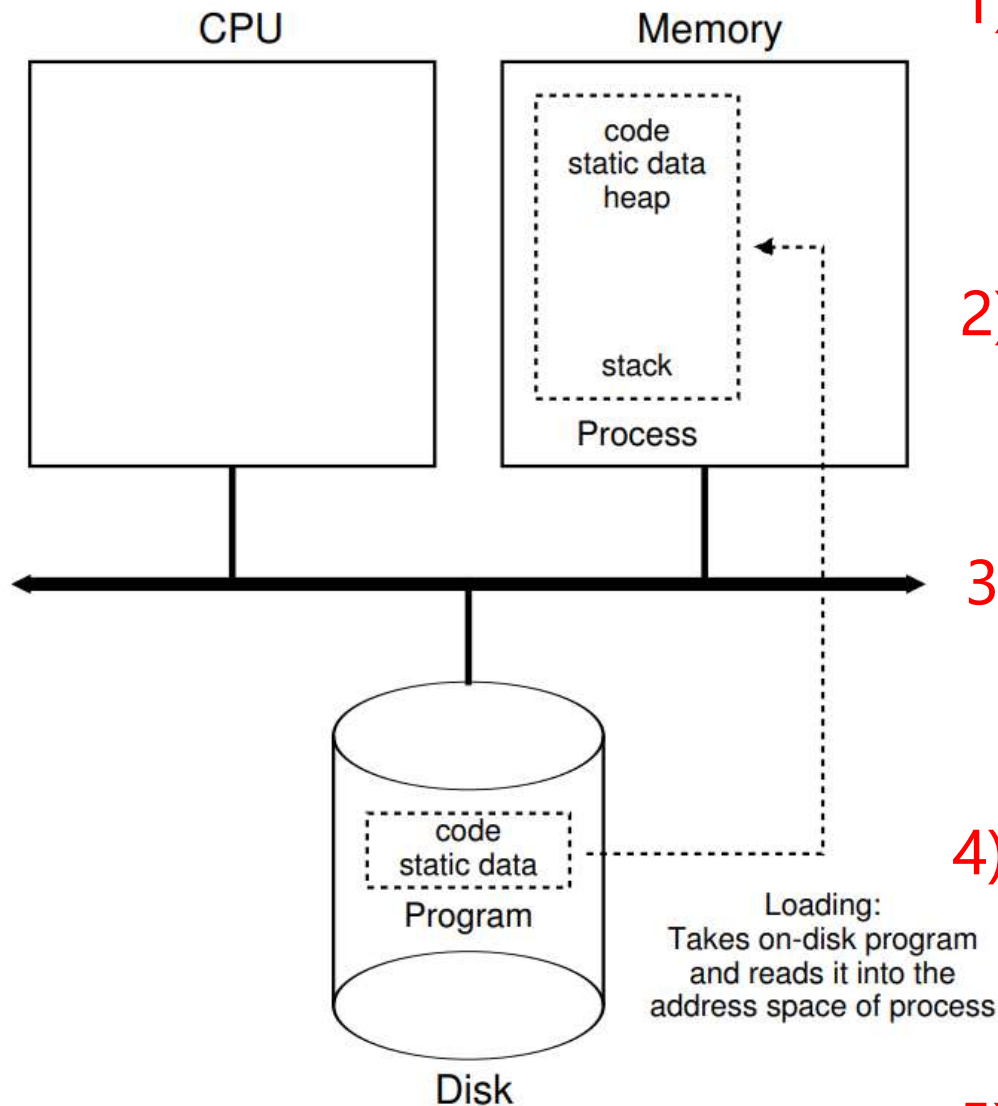


Process Creation

- ▶ Assign a unique process identifier
- ▶ Allocate space for the process
 - ▶ Private user address space (program and data), stack, and heap
 - ▶ Whether to share any existing address space? If yes, set up the appropriate linkages.
 - ▶ Allocate the space for a process control block.
- ▶ Initialize process control block
- ▶ Set up appropriate linkages
 - ▶ Ex: add new process to linked list used for scheduling queue
- ▶ Create or expand other data structures
 - ▶ Ex: maintain an accounting file (for billing and/or performance assessment)



Transformation from Program to Process



1) Loading code and static data from program to process

Initially, program is in executable format on disk

2) Memory allocation for stack

local variable, function parameters and return address

3) Memory allocation for heap

Explicitly requested dynamically allocated data

4) I/O-related initialization tasks

For example: the three open file descriptors in UNIX

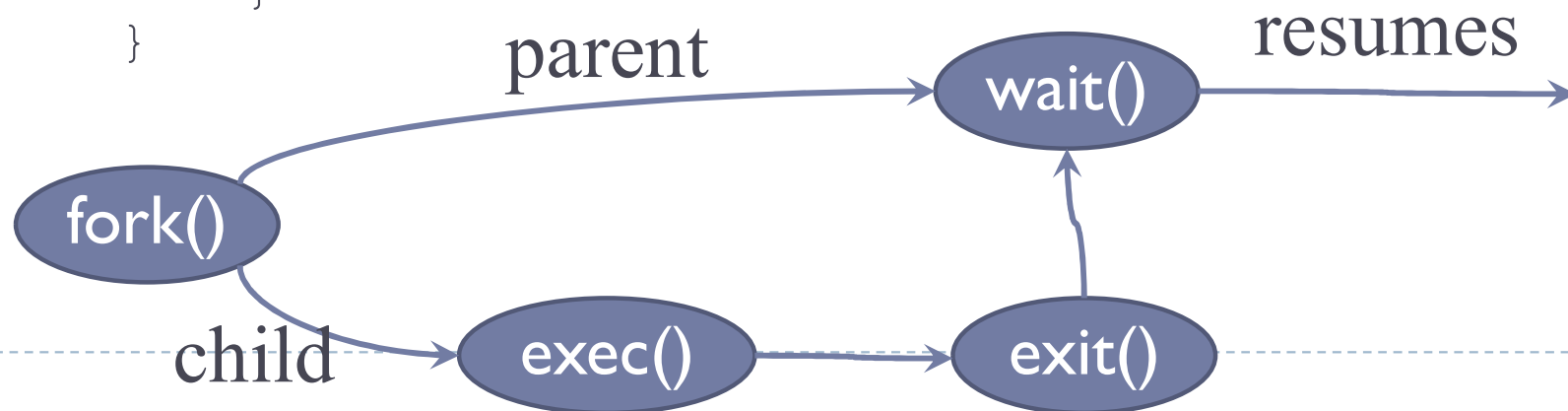
5) Start the program at entry point

A Simple Example in Unix

```
int main() {  
    pid_t pid;  
    pid = fork();  
    if (pid < 0) {  
        fprintf(stderr, "Fork Failed");  
        exit(-1);  
    }  
    else if (pid == 0) {  
        execlp("/bin/ls", "ls", NULL);  
    }  
    else {  
        wait(NULL);  
        printf("Child Complete");  
        exit(0);  
    }  
}
```

The pid value returned to the child process is 0.

The parent gets an integer value larger than 0.



The Separation of `fork()` and `exec()`

Why?

- ▶ Their separation is essential in building a UNIX shell, and lets the shell do some work after the call to `fork()` but before the call to `exec()`.

- ▶ For example, the **output redirection**:

```
prompt> wc p3.c > newfile.txt
```

Shell使用`fork()`创建一个子进程，然后关闭掉它的标准输出，并打开文件`newfile.tx`文件，然后再用`exec()`来执行`wc p3.c`

- ▶ Another example, the **pipe mechanism**:

```
grep -o foo file | wc -l
```

Shell使用`pipe()`系统调用，并将第一个进程的输出生连接到内核中的管道，而另一个进程的输入被连接到同一个管道



Process Termination

- ▶ When a process finishes executing its final statement, it terminates.
 - ▶ It asks OS to delete it by using the `exit()` system call.
 - ▶ The process may return a status value to its waiting parent process (via the `wait()` system call).
 - ▶ All the resources of the process are deallocated and reclaimed by OS – including physical and virtual memory, open files and I/O buffers.
- ▶ Other situations for process termination:
 - ▶ A process can cause the termination of another process via a system call (Usually called by the parent of the process to be terminated).



Modes of Execution (1)

执行模式

These instructions are called Supervisor, privileged, or protected instruction

- ▶ Most processors support at least two modes of processor execution
 - ▶ User-mode (用户模式): the less-privileged mode
 - ▶ Supervisor Mode, System Mode, Control Mode, or Kernel Mode (监督模式、系统模式、控制模式或者内核模式): the more-privileged mode
- ▶ What is the difference?
 - ▶ Certain instructions can only be executed in the more-privileged mode.
 - ▶ Reading or altering a control register, such as the program status word
 - ▶ Primitive I/O instructions
 - ▶ Instructions related to memory management.
 - ▶ Certain memory regions can only be accessed in the more-privileged mode

Modes of Execution (2)

- ▶ Why use two modes?
 - ▶ It is necessary to **protect** the operating system and key operating system tables from interference by user programs
 - ▶ It is also necessary to **prevent unauthorized access** of resources by user programs.



Modes of Execution (3)

- ▶ A process executing in user mode should **be able to switch the processor to supervisor mode**, in order to begin executing OS code
- ▶ Whenever the processor switches to supervisor mode, the computer should **only execute OS code**
- ▶ Whenever user mode software calls the OS, the processor should be switched into supervisor mode

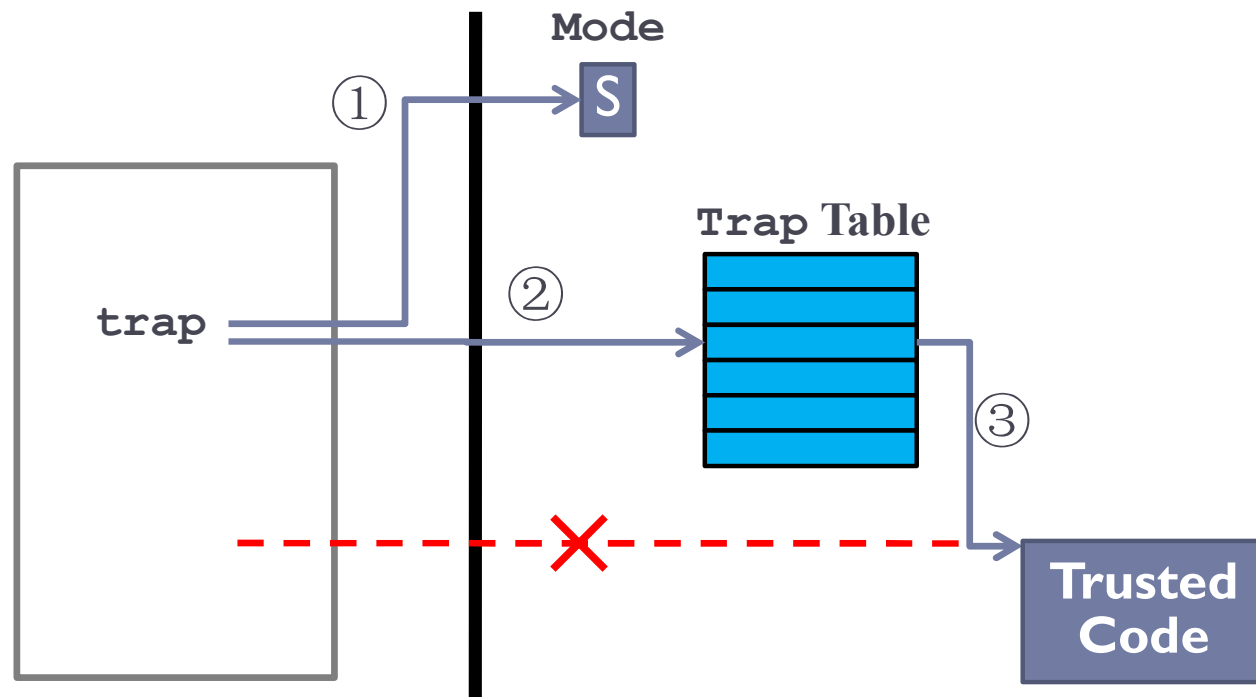


Modes of Execution (3)

Choices of Execution Modes

- ▶ Computer boots up in supervisor mode.
 - ▶ Used by bootstrap and OS to initialize the system
- ▶ Applications run in user mode
 - ▶ Operating system changes to user mode before running user code
 - ▶ Applications have no way to get into supervisor mode
 - ▶ Instructions that change the processor status register are privileged.
- ▶ Reentering supervisor mode is strictly controlled.
 - ▶ **Only** happens in response to **traps** and **interrupts**





Mode Switching (模式切换)

Mode switching is switching between kernel mode and user mode.

When switch to kernel mode? Three key scenarios:



System Calls (主动请求)

用户程序主动发起服务请求，如文件读写 (read/write)、进程创建等，主动陷入内核态。



Exception Handling (被动中断)

程序执行错误触发，如除零运算、缺页异常或非法指令，迫使CPU进入内核处理。



Interrupt Handling (硬件中断：异步事件)

外部设备信号触发，如键盘输入、定时器中断或网卡数据到达，打断当前进程执行。

性能洞察：成本与权衡

直接开销仅需**几十到几百个时钟周期**，但KPTI等安全机制会导致TLB缓存失效，增加内存访问的间接延迟，这是系统安全与性能之间的典型权衡。

Mode Switching (1)

- ▶ In the interrupt cycle, the processor checks to see if any interrupts have occurred
 - ▶ If no interrupts are pending, the processor proceeds to fetch cycle.
 - ▶ If an interrupt is pending, the processor do the following:
 - ▶ **Saving the context** of the currently running program
 - ▶ **Setting the program counter** to the starting address of an interrupt-handler program
 - ▶ **Switching from user mode to kernel mode** so that the interrupt processing code may include privileged instructions



Mode Switching (2)

- ▶ What constitutes the context that is saved?
 - ▶ It must include any information that may be altered by the execution of the interrupt handler and that will be needed to resume the program that was interrupted.
- ▶ In most operating systems, *the occurrence of an interrupt does not necessarily mean a process switch.*



When Switch to Kernel Mode? (1)

- ▶ Possible events that may give control to OS.
 - ▶ **Interrupt (or Hardware Interrupt)**: An interrupt is due to some sort of event that is **external to and independent of** the currently running process
 - ▶ **Trap**: A trap relates to an **error** or **exception** condition generated **within** the currently running process
 - ▶ **System calls (or Software Interrupt)**: The program being executed can make a system call explicitly, to request a system function or service (such as a file open) from OS



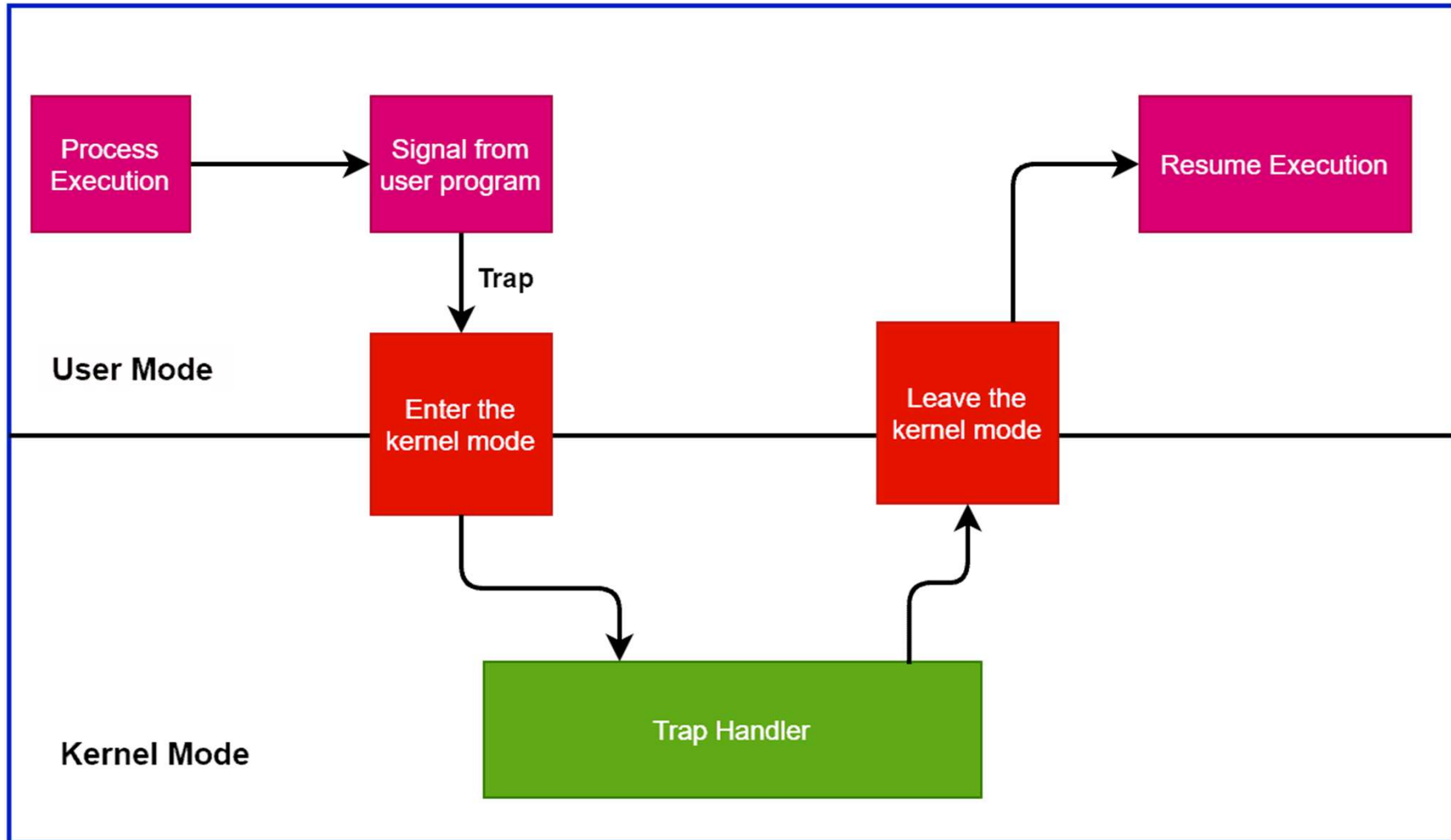
When Switch to Kernel Mode (2)

Examples

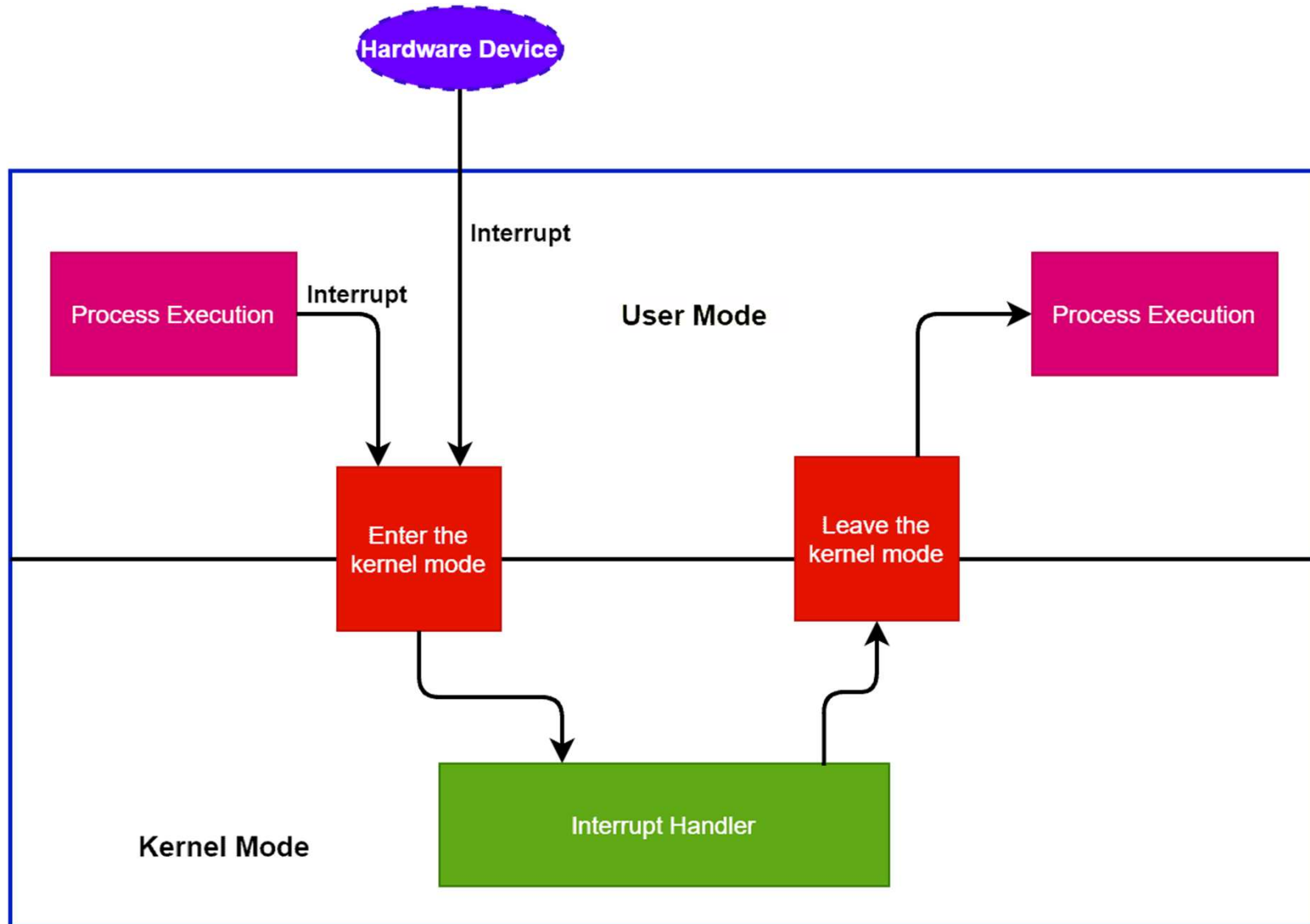
- ▶ Clock interrupt
 - ▶ Whether the currently running process has executed for the maximum allowable time slice (**if yes, doing process switch**)
- ▶ I/O interrupt
 - ▶ What I/O action has occurred? If it constitutes an event, then the OS moves the corresponding blocked processes to the Ready state. The OS has to determine whether to continue executing the currently running process, or **to preempt that process for a higher-priority Ready process.**
- ▶ Memory fault
 - ▶ A virtual memory address reference is not in main memory, so the corresponding block must be brought into main memory (**It is a I/O request**)



Traps



Interrupt



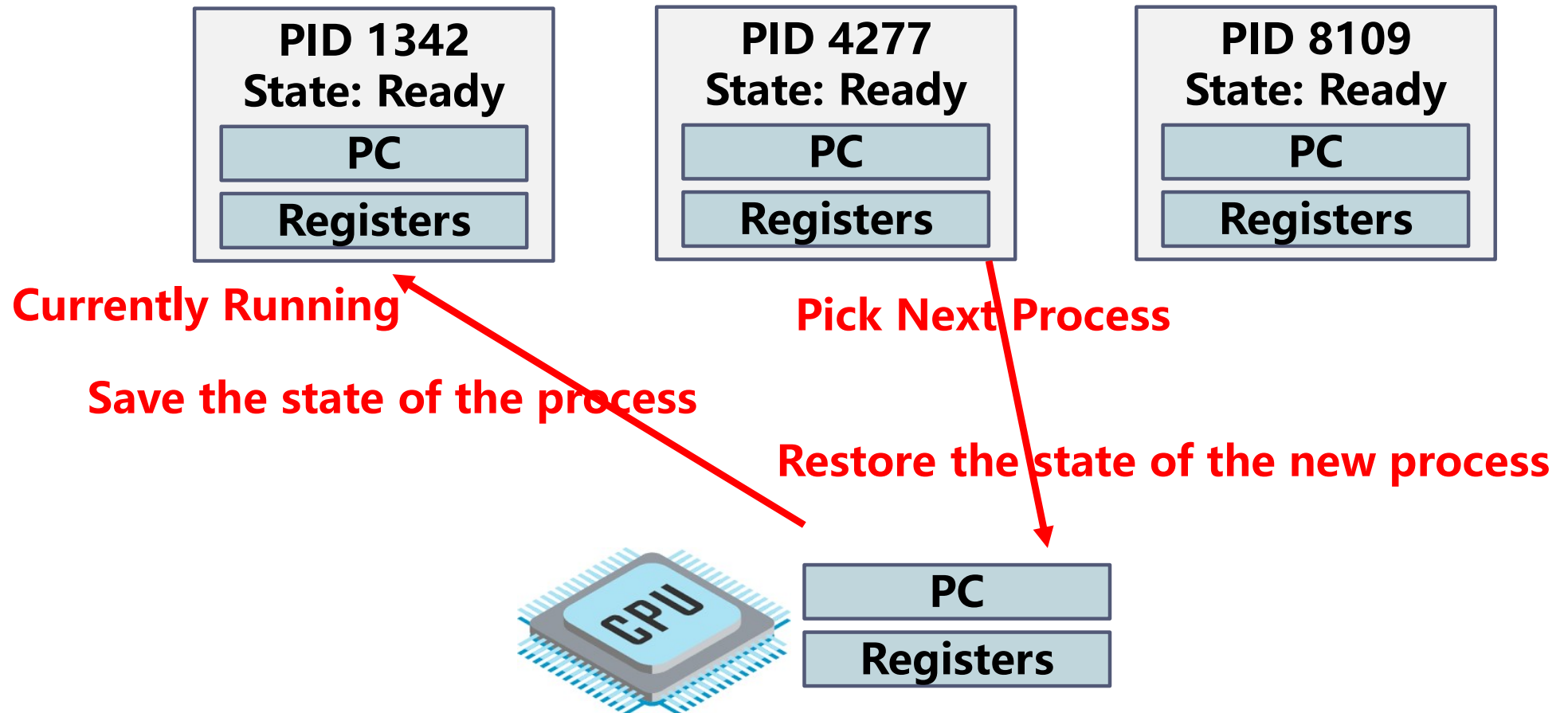
Context Switching: Definition

上下文切换

- ▶ What is context switch?
 - ▶ Storing the state of a process (or thread), and then restoring the state of another process (or thread) and starting its execution
- ▶ Context switch is an essential feature of a multiprogramming or multitasking OS
 - ▶ It allows multiple processes (or threads) to share a single CPU



Context Switching



Process Switching

Definition

A running process is interrupted and the operating system assigns another process to the Running state and turns control over to that process

▶ Three issues

- ▶ What events trigger a process switch?
- ▶ What is the difference between mode switching and process switching?
- ▶ What must the OS do to achieve a process switch?



Steps of a Process Switching

- 1) **Save the context** of processor including program counter and other registers
- 2) **Update the process control block** of the process that is currently running
- 3) Move the process control block to **appropriate queue** (Ready, Blocked on Event i , or Ready/Suspend)
- 4) **Select another process** for execution
- 5) Update the process control block of the process selected
- 6) Update **memory-management data structures** (depending on how address translation is managed)
- 7) **Restore the context** of the selected process

